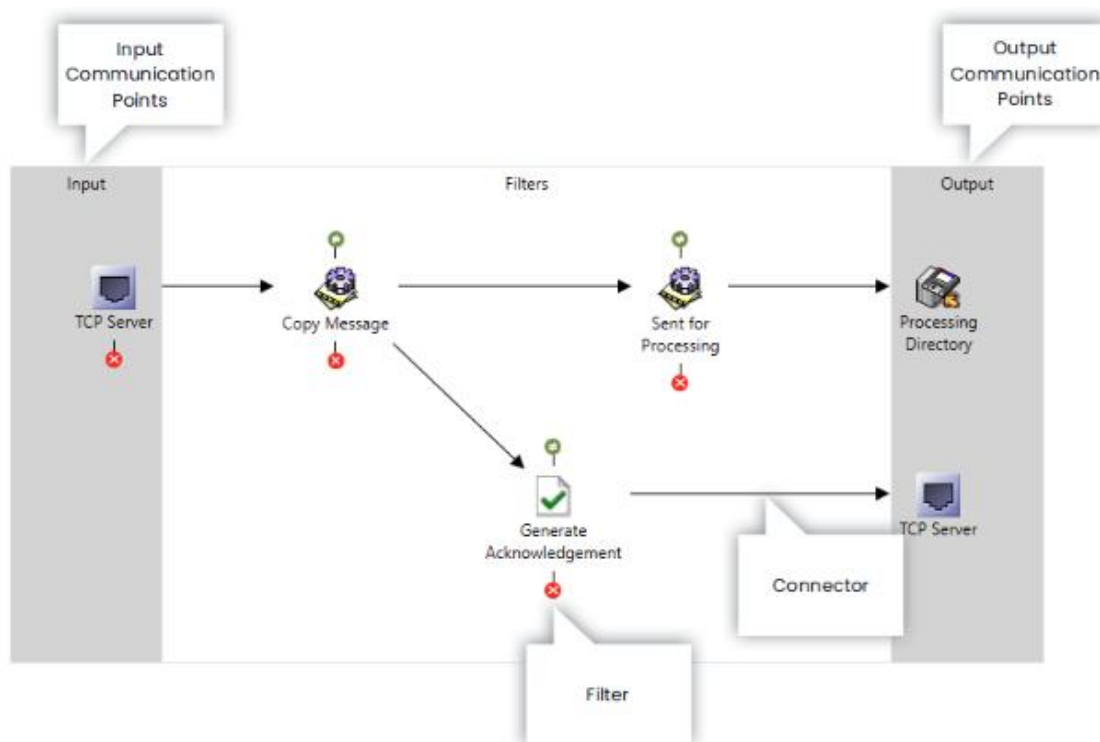


Route Requirements

Routes and communication points are created and organized in the IDE **Workspace**. They are then assembled into a working solution on the Route Canvas.

Rhapsody routes contain the following:

- One or more input communication points that receive messages from an external system or systems for processing by the route.
- One or more filters to provide the business logic for the route. An exception is the pass-through route, which passes messages directly from an input to an output communication point without making any changes.
- One or more output communication points that pass processed messages from the route to an external system or systems.
- Connectors to link route components and define the path taken by messages as they move through the engine. Connectors might also apply conditions to restrict movement of messages across the connection to only those that match specified criteria.



An example route illustrating the various components.

Best Practice

Before working on a new solution, you should develop a high-level overview of its intended purpose and establish a naming convention for its lockers, folders, routes

and communication points. This helps identify which components belong together and assists when searching for a specific item.

All components should be labeled in a clear manner and indicates their purpose.

Lockers and Folders

A Rhapsody solution begins with deciding which locker will contain its routes and communication points. This can be an existing locker, or one created specifically for the solution. Organizing these components into lockers and folders offers the following advantages:

- A hierarchy of folders and subfolders is used to organize and group related components. This helps make the configuration easier to understand and maintain.
- Security rules can be applied to each of the lockers. These rules are used to manage access to the configuration and the messages processed by each locker. This is important where different access rules are required for different parts of the configuration.
- Checking in, starting, stopping and generating the documentation for components in a locker or folder can be performed as a single action by right-clicking on the locker or folder.

In most Rhapsody solutions, you will use very few lockers, and instead use a hierarchy of folders to organize your configuration.

- Lockers should be named in a way that clearly identifies the solution or project they are part of.
- Folders should be named to summarize the function of the components in the folder. Remember that folders can contain sub-folders, allowing a set of components to be grouped within a larger solution.

Working with lockers

To create a locker, right-click the engine name in the **Workspace** and select **New Locker**, then give it a unique name.

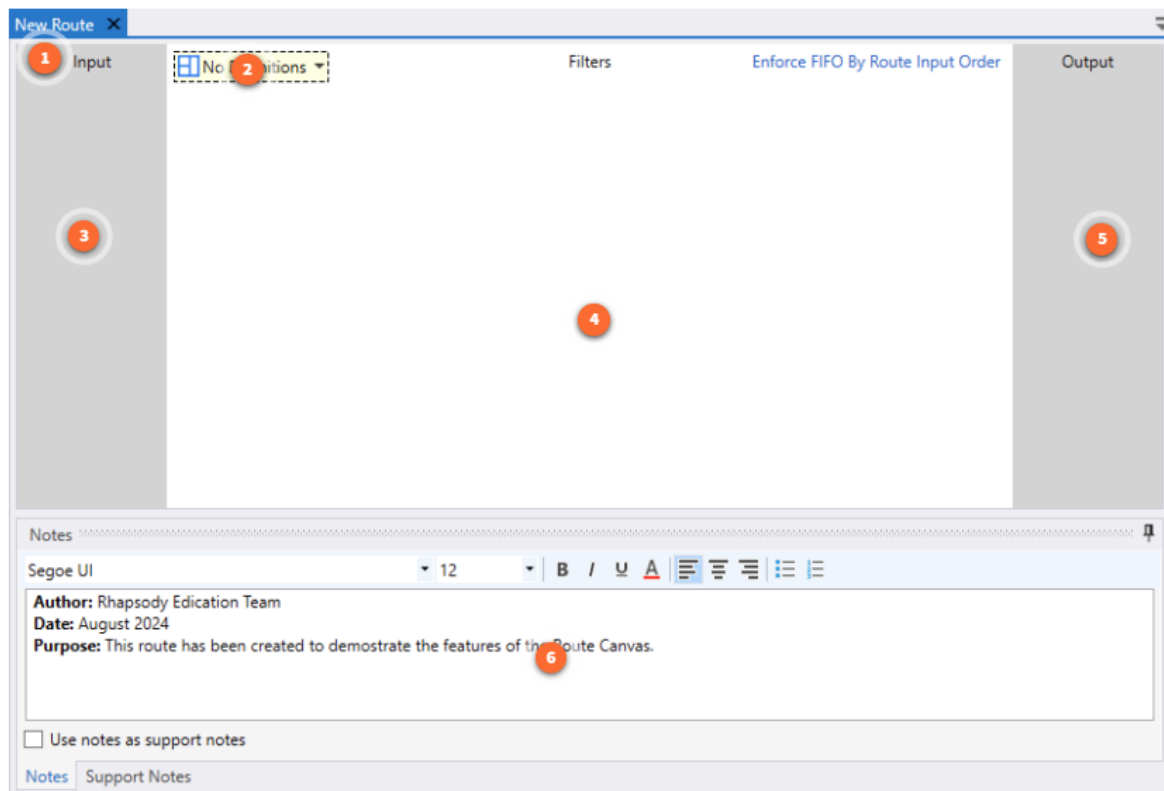
- Lockers exist at the top level only; sub-lockers cannot be created.
- Lockers can be renamed, and any changes must be checked in before they take effect. The same is true when deleting a locker.
 - Routes, communication points and message and mapper definition files cannot be shared between lockers. However, they can be copied from one to another using drag and drop or from their right-click context menu.

- Copied components, including definition files, must be checked in before they can be used.
- Rhapsody variables are not linked to a specific locker and can be used in the configuration of any component regardless of the locker it is contained in.
- Messages can only be routed from a route in one locker to a route in a second locker using the Dynamic Router communication point. This component is described further in the "Route Building" module later in this course. While lockers do not add any noticeable performance overhead, a configuration should not be split between lockers unless there is a need to restrict user access to specific parts of it.
- Lockers should always be distinctly named to identify the project(s) they are associated with.
- **Locker security**
- Security can be applied to each locker in Rhapsody to control how different groups of users can modify a configuration or view the messages being processed. Configuring access to lockers and folders, and the routes and communication points they contain is managed in the IDE User Manager.
- **Working with folders**
- A new folder can only be created inside a locker; however, multiple sub-folders levels are permitted. Folders at the same level must have unique names. You will probably find it useful to use folders liberally in your Rhapsody configuration.
-
- The 'red tick' alongside the folder or locker icon identifies it as a new or renamed component that is awaiting check-in.
- Use the right-click menu option, or press **Delete**, to delete a folder. You are asked to confirm this action because the locker or folder might contain items that would also be deleted. Deleting a locker or folder is not complete until the component has been checked in.
-
- The 'red cross' next to the folder or locker icon means it (and its contents) have been marked for deletion.

Creating a New Route

The route editing canvas

You create a route on the route editing canvas using communication points, filters and connectors.



Message flow

- The flow of messages through a route is from **Input** to **Output** - from the left side of the canvas to the right.
- A route might contain multiple input and/or output communication points, with the connections between them defining the path taken by messages as they are processed by the engine.
- Connectors might include one or more filters to modify the message's structure and/or content. Criteria can be applied to a connector to restrict the messages that can pass, with non-matching messages sent to an alternate connector.

The Input Communication Point

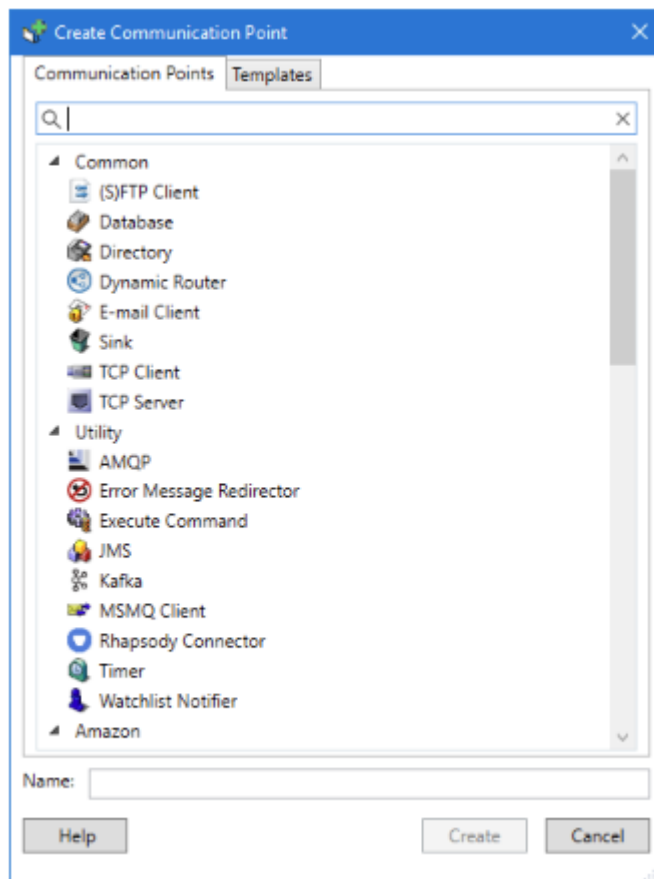
The Input communication point manages the communication protocols associated with their connections to external systems and accepts messages from those systems to pass into the route for processing.

To create a communication point, right-click the locker or folder in the **Workspace** where you want to add the communication point and select **New Communication Point**. You can also select the **Add** button from the **Workspace** toolbar.

At any time, you can drag a communication point into a different folder in the same locker. This creates a copy of the component in the new location.

Create Communication Point dialog

After you add the communication point, the **Create Communication Point** dialog opens listing all available communication points grouped by type.



Creating a Directory communication point

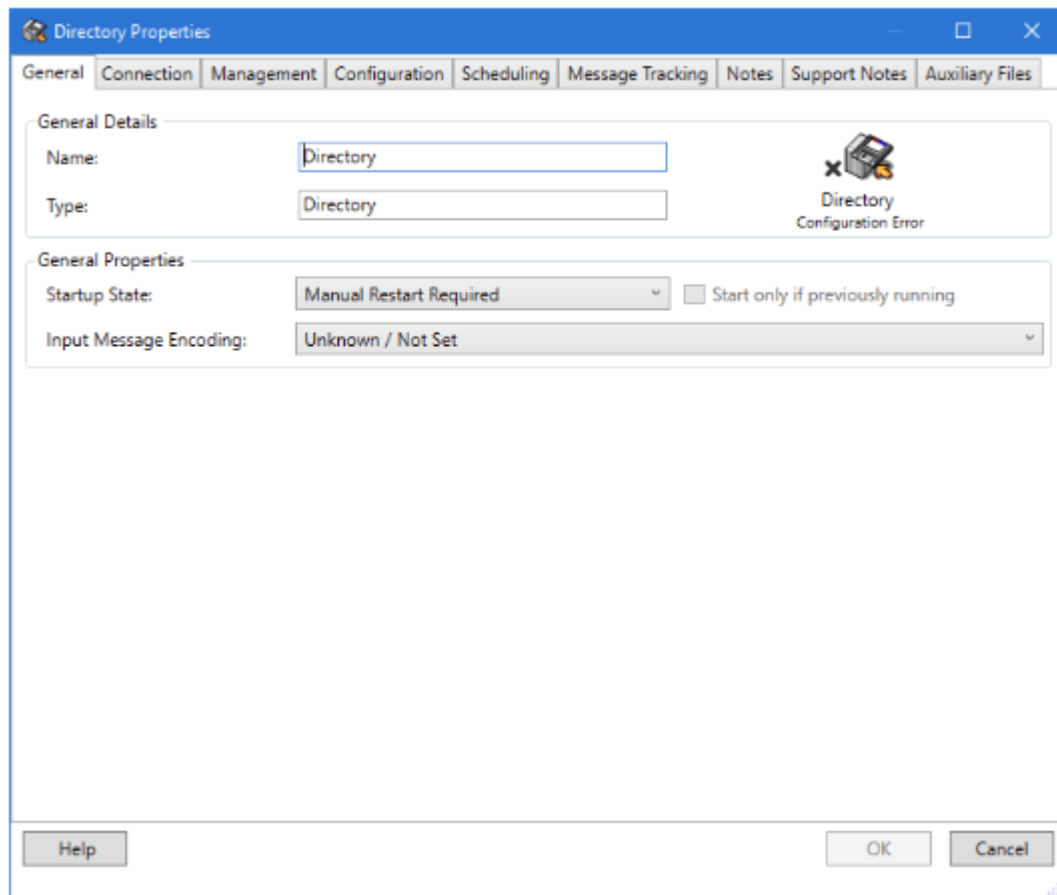
For this example, we'll select **Directory**. This communication point accepts messages placed into a specified directory then passes them into the route for further processing.

Provide a meaningful name for the communication point and click **Create**.

The name you use could identify the communication point's **Type** (although this can usually be determined from its associated icon), **Mode** (whether it receives or sends messages, or both) and **Purpose** (such as the project or solution it will be part of). Using spaces in a name rather than underscores improves readability and the overall layout of the route.

Communication point properties - General tab

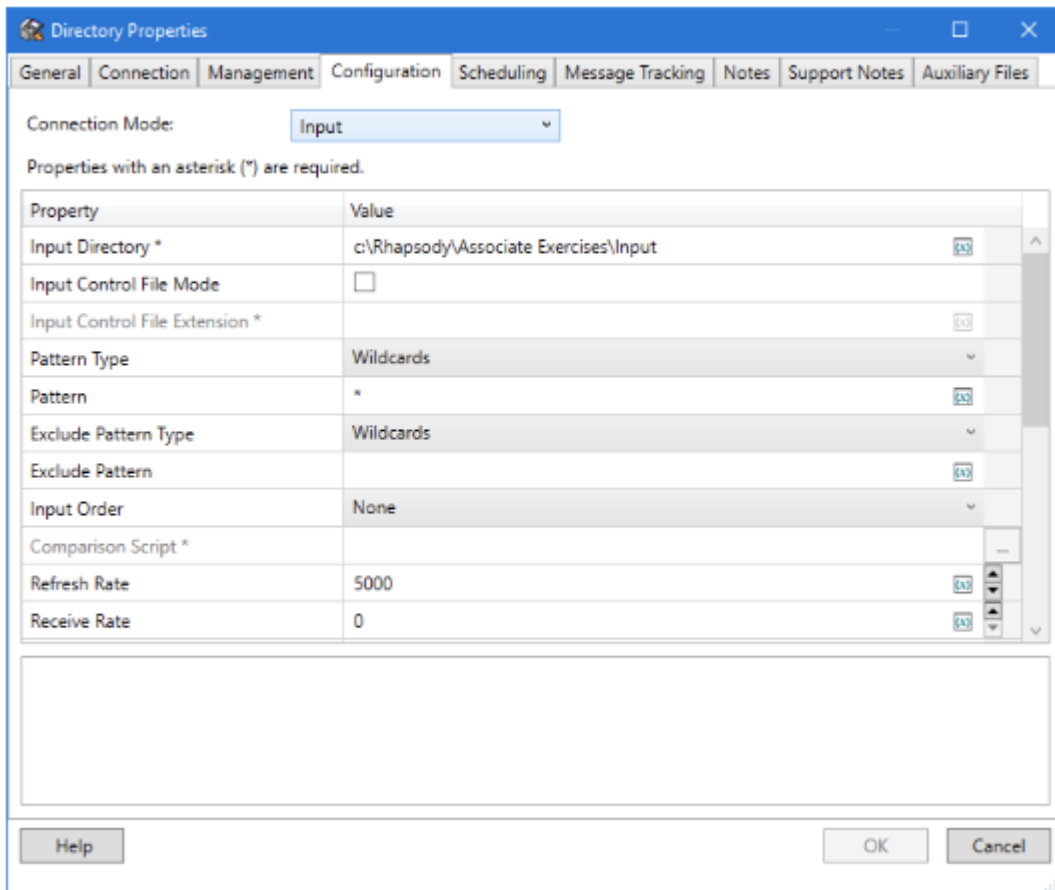
Rhapsody automatically opens the **General** tab of the **Properties** window for a newly created communication point. The communication point's name can be edited but its type is read-only.



Communication point properties - Configuration tab

How the Directory communication point interacts with messages is set on the **Configuration** tab.

By default, a Directory communication point's **Connection Mode** is set to **Bidirectional**, giving it the ability to both send and receive messages. For this example, it is set to **Input** to restrict it to receiving messages only. The other options on this window are described in the "Communication Points" module later in this course.



Contextual help for a selected property is displayed at the bottom of the dialog, with more extensive assistance available by selecting **Help**.

Entering values

A configuration property's **Value** field can be populated using a variety of methods, depending on the type of property:

- Many properties are configured by entering text. The content of a message property is specified as: \$(<property_name>). A Rhapsody variable appears as: \$(<variable_name>). The Variables icon to the right of the field can be used to select a variable from a list.
- When the property is selected from a list of values, a dropdown is used. Select the field to display the available choices.
- Some fields require more complex logic for selecting a value. These use a popup that is accessed by selecting the ellipsis (...) to the right of the field.

The Output Communication Point

The Output communication point passes messages whose processing has been completed on to one or more external systems. Output communication points are created in a similar way to Input communication points.

Output-only communication point examples

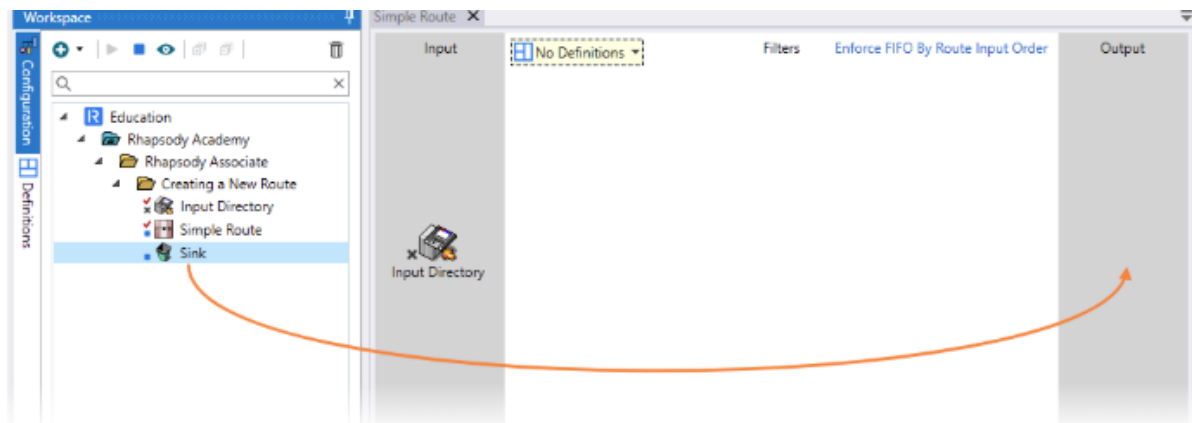
Most communication points can be configured as Input, Output or Bidirectional, which means they can both send and receive messages. A small number of communication points including the following are Output-only:

- **Sink**
Discards messages without further processing. Before this happens, a copy is saved to the Rhapsody Archive, where it can be viewed in the Management Console. This communication point has no configurable properties.
- **Execute Command**
Delivers the message to an application or script on the Rhapsody server.

Creating Connections

To add a communication point and filter to a route, drag their icon from the **Workspace** and **Filter** toolbar, respectively.

- Rhapsody does not allow a communication point configured as Input to be placed on the Output side of a route, and vice-versa.
- Bidirectional, In->Out, and Out->In communication points are typically placed on both the Input and Output sides of the route.
- The same communication point can appear on two or more routes, provided it is in the same locker as the route. This is typically only done with communication points in Output mode.

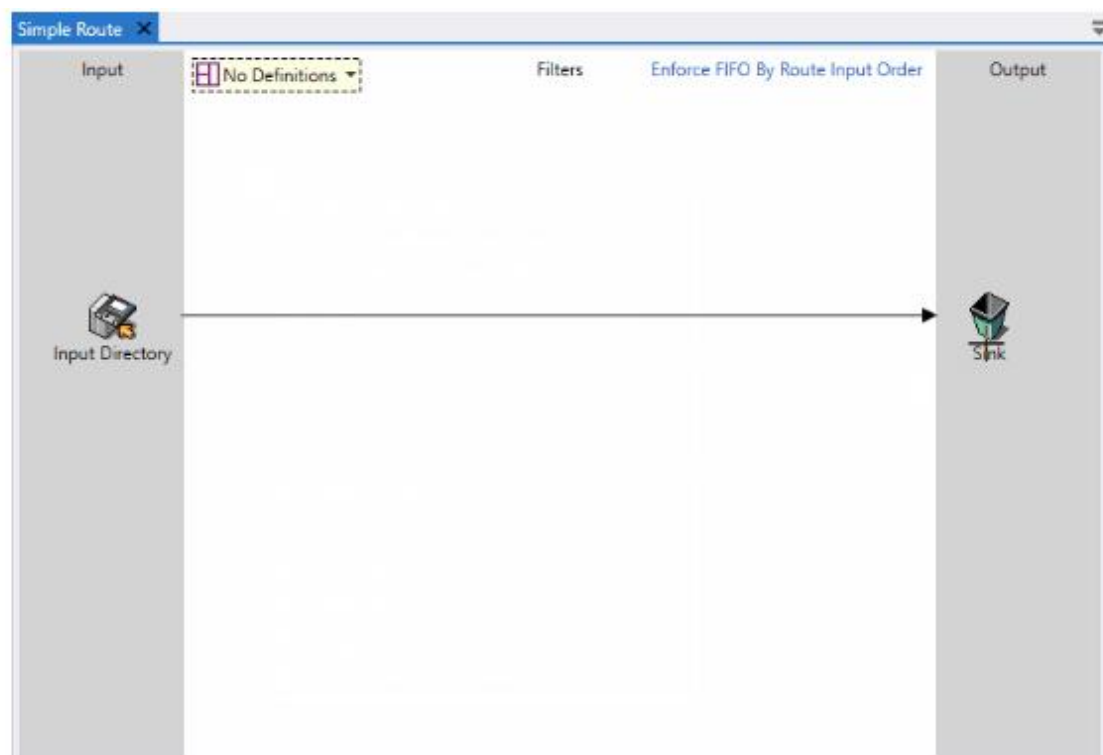


The Connector tool

The route pipeline is defined by adding connectors to all components. These connectors define the pathway that messages follow from the Input to the Output communication points. Filters might be present to apply business logic to the messages as they are processed by the engine.

To add a connector

1. Right-click the route and select **Draw Connectors**. (The icon to create a connection can also be found on the toolbar).
2. Move your mouse cursor over the Input communication point until a blue border appears
3. Press and hold down the mouse button and move the cursor to the Output communication point.
4. When the blue border appears around it, release the mouse button to add the connector to the route.



Animation showing the creation of a connection.

After you have added connections to your route, press the **Esc** key or select the pointer icon from the toolbar. This will restore your mouse pointer and allow you to realign or add components to your route.

If the connection is not perfectly horizontal (and you would prefer it was), select either communication point and drag it up or down. When nearly horizontal, it will snap into alignment.

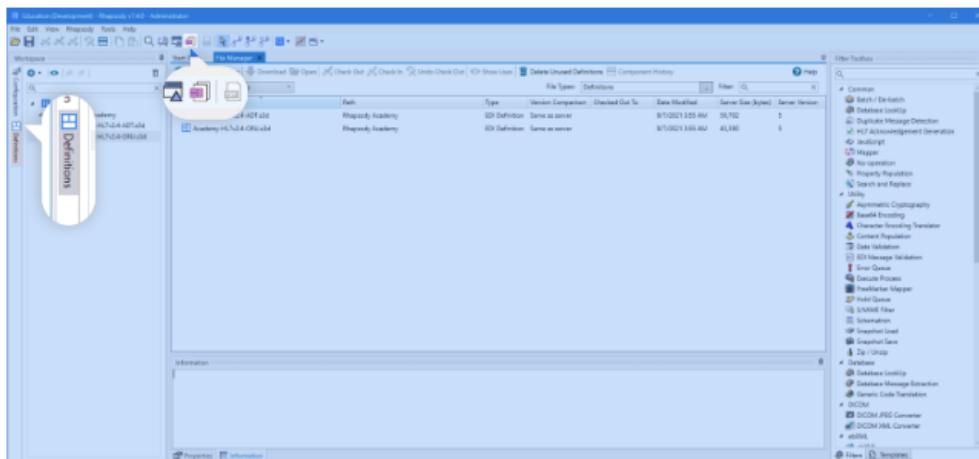
Message Definitions

When you need to interpret a message, a Message Definition must be loaded and associated with your configuration. Validating the structure of the message enables Rhapsody to:

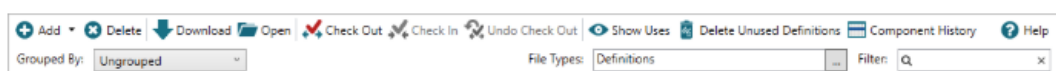
- Conditionally process a message based on content.

- Generate acknowledgements.
- Extract details about a patient.
- Transform messages to other formats.
- Extract parts of the message for injection into a database.

Message Definitions are managed on the **Definitions** tab on the left-hand side of the **Workspace**, or by selecting the **File Manager** icon in the IDE toolbar to display a tab listing all the definitions in the configuration.



We will use message definitions in Exercise 103b and look at them in more detail in the "Interpreting Messages" module. For now, all you need to understand is that these definitions are managed through the **File Manager** and perform message parsing and validation functions.



File manager buttons.

Checking In Your Work

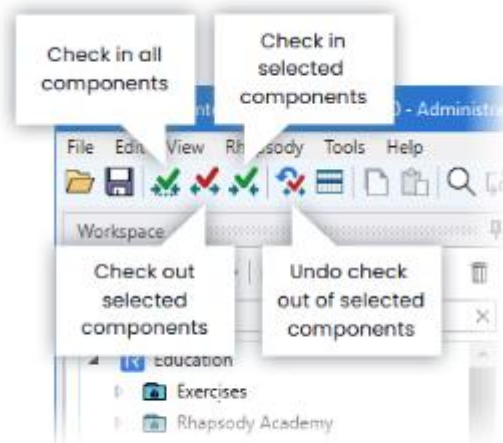
As you complete your changes, you will need to perform a check-in to commit the changes to the Rhapsody server. When the check-in completes, the updated configuration becomes active and starts processing messages.

A change to a filter's configuration is checked in by checking in the route that it is part of. Communication points can be checked in individually. If you try to log out of Rhapsody while there are changes that have not been checked in, you will be prompted to do so. You can, however, choose to log out without checking in the changes. If you do this, your changes will be lost.

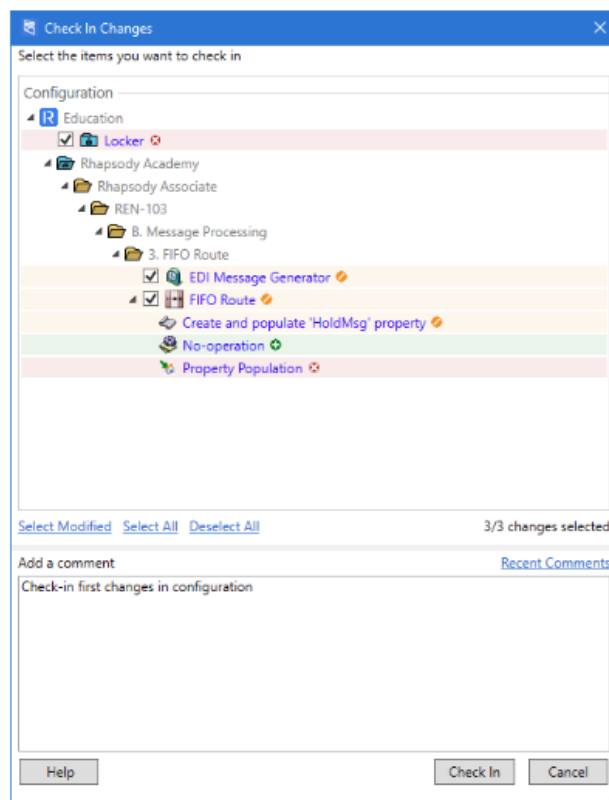
We recommend that each check-in relates to no more than one logical change to the configuration. The check-in should occur when the components have been created or amended and are ready for testing.

Performing a Check In

To perform a check-in, select the **Check In** or **Check in all components** button on the toolbar, or right-click a component and select the option.



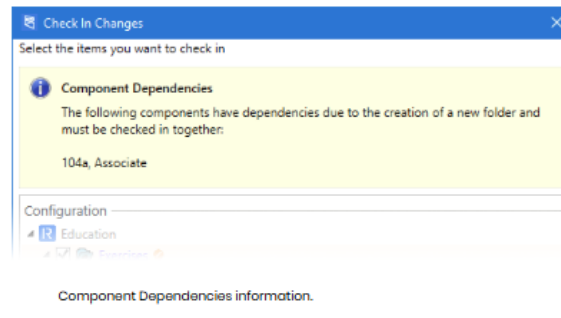
Check In buttons on the toolbar.



Check In window.

On the **Check in Changes** window, the **Check in all components** option will automatically select all checked out components. The **Check In** option will select just the chosen component and any dependencies (for example, a newly created definition the component relies on). This screenshot shows the modification, addition, and deletion of several components that are about to be checked in.

From the **Check In Changes** window, you can add or remove components and include a comment that describes the changes. If you remove a component that is a dependency for other selected components, those components will also be deselected with a note in the dialog indicating what components must be checked in together.



Check-in comments

You can add comments to describe the changes. Comments could provide information on the reason for the change or, in a production environment, the name of the person who approved the change and the number of the support ticket associated with the change. It is recommended that a comment be associated with every check-in. Comments make it easier to review the reasons for a particular check-in.

Check-in comments can be set as optional or mandatory. When mandatory, rules can be created to require that the comment contain certain details, for example, a ticket number.

Undo check out

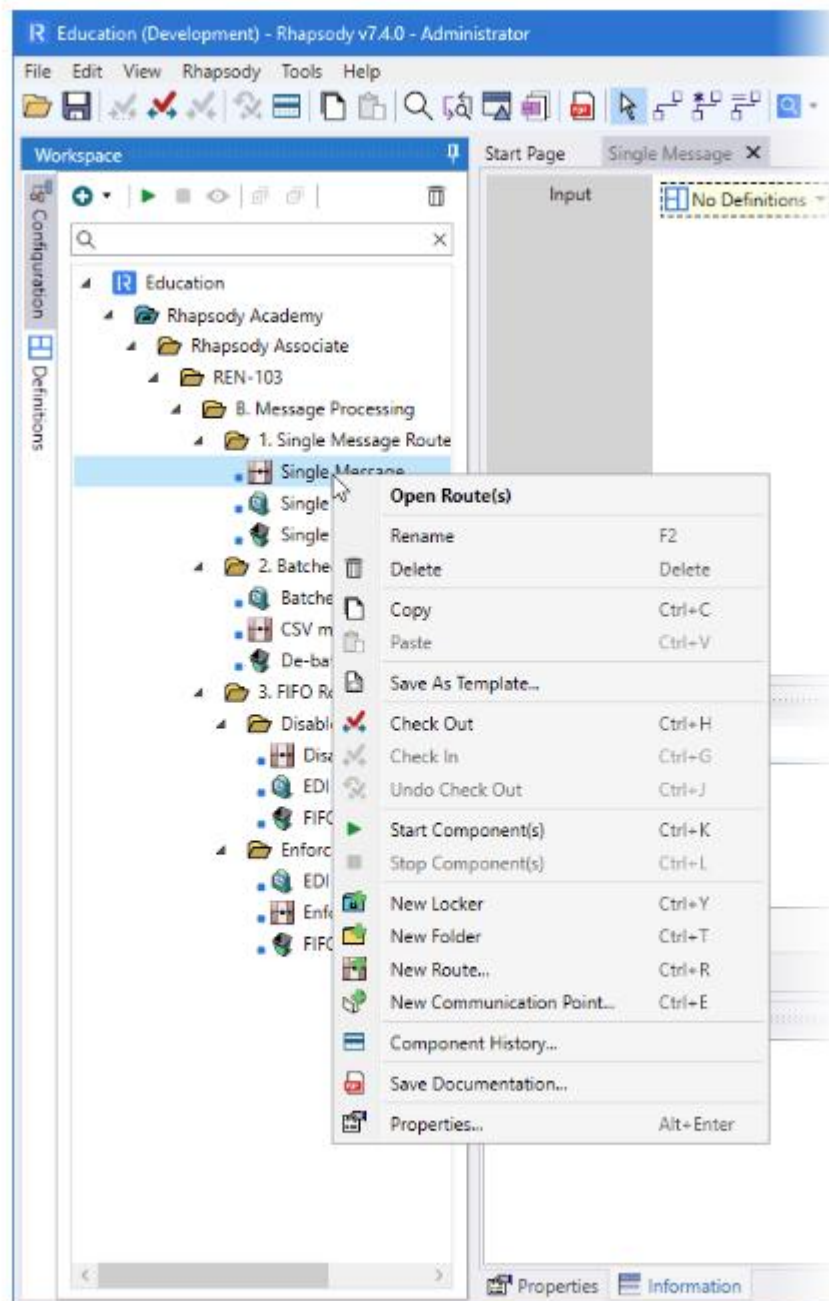
Before completing a check-in, you can undo all changes with the **Undo Check Out** option. This option is available from the component's context menu or the menu bar when the component is selected. Selecting this option copies the component's configuration from the engine back to the IDE, overwriting any changes and restoring the component's configuration to the state it was in prior to its check-out.

Component Status

There are two actions that must be performed on a route and its component filters and communication points before they can begin processing messages.

1. All components must be checked in. This copies the component's configuration from the IDE to the engine, from where message processing is controlled.
2. All components must be started. This activates the component and enables it to begin message processing.

Right-click a component to view the component's status.



IDE Workspace.

Component State Indicators

The component status options, and their effect on message processing, is shown below.

The Information panel

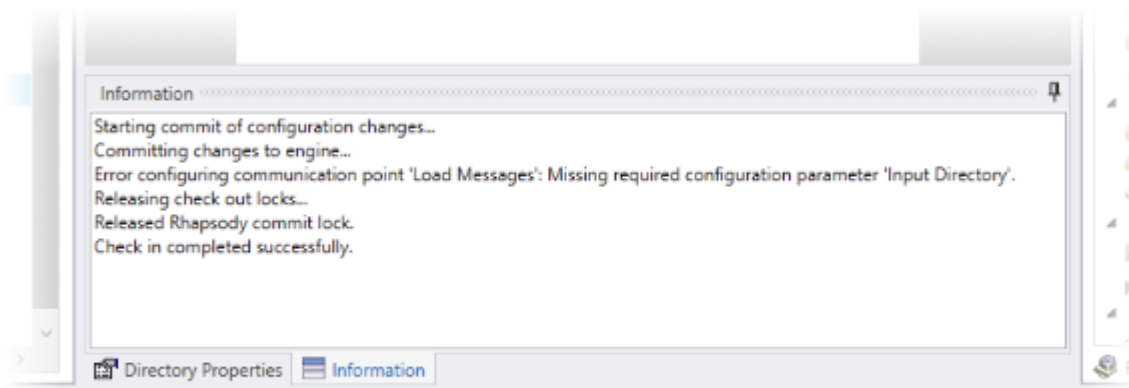
The contents of the **Information** panel below the **Route Editing Canvas** are dependent on either the currently selected component in the editing canvas or the last completed action in the engine.

- **Properties**

Shows the configuration of the currently selected communication point or filter.

- **Information**

Lists the most recently completed actions. When an error occurs, it is reported in the panel. The following screenshot shows the check-in of a communication point. Notice the error that has been reported because of a missing configuration in the component.



Information panel showing the check in of the Load Messages communication point.

Exercise - Building a Simple Route

This exercise guides you through creating a simple route and then using it to process messages. This route includes the following two communication points and a connector:

1. 1

Input Directory

An Input communication point that accepts messages and passes them into the route.

2. 2

Connector

Passes messages from the Input side of the route to the Output side.

3. 3

Output Sink

An Output communication point that receives messages from the route and silently discards them.

Resources

This exercise uses the messages in the *.Associate Files and Definitions\103 - Building a Simple Route* folder in the resources ZIP file available at the start of the course.

Setup

Create a locker to hold your working routes and communication points. This ensures that your work is kept separate from other configuration on the engine. Throughout this course we will use the **Exercises** locker to illustrate work performed in the various exercise.

We recommend that specific folders and subfolders are created within this locker to contain the routes and communication points relating to each of the Associate exercises. Working folders should be named in a way that clearly identifies the exercise they are part of.

Create a locker

1. 1

Right-click any component in the IDE **Workspace** and select **New Locker**.

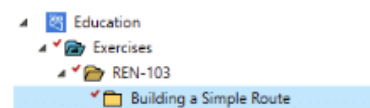
2. 2

Provide a name (we recommend **Exercises**) and press **Enter**. Press the **F2** function key to rename a locker, if necessary.

Create a folder

Right-click the locker you have created for the exercises then select **New Folder**. Subfolders can be created from the parent folder's context menu. Both can be renamed by selecting the folder to rename and pressing **F2**.

It is recommended that you create a top-level folder in your locker for each module in the course. Subfolders can be created underneath it. For the first exercise, name the folder *A. Building a Simple Route*. Make sure your names are both meaningful and consistent.



Create the route

You can create routes and communication points in any order. However, creating the route first might help you visualize the configuration as it is built. In this exercise, we will create a route first. Use the image below the listed steps for guidance.

1. 1

Right-click the subfolder you created for this exercise and select **New Route** from the context menu.

2. 2

Name the route (we suggest: **Simple Route**) and press **Enter**.

3. 3

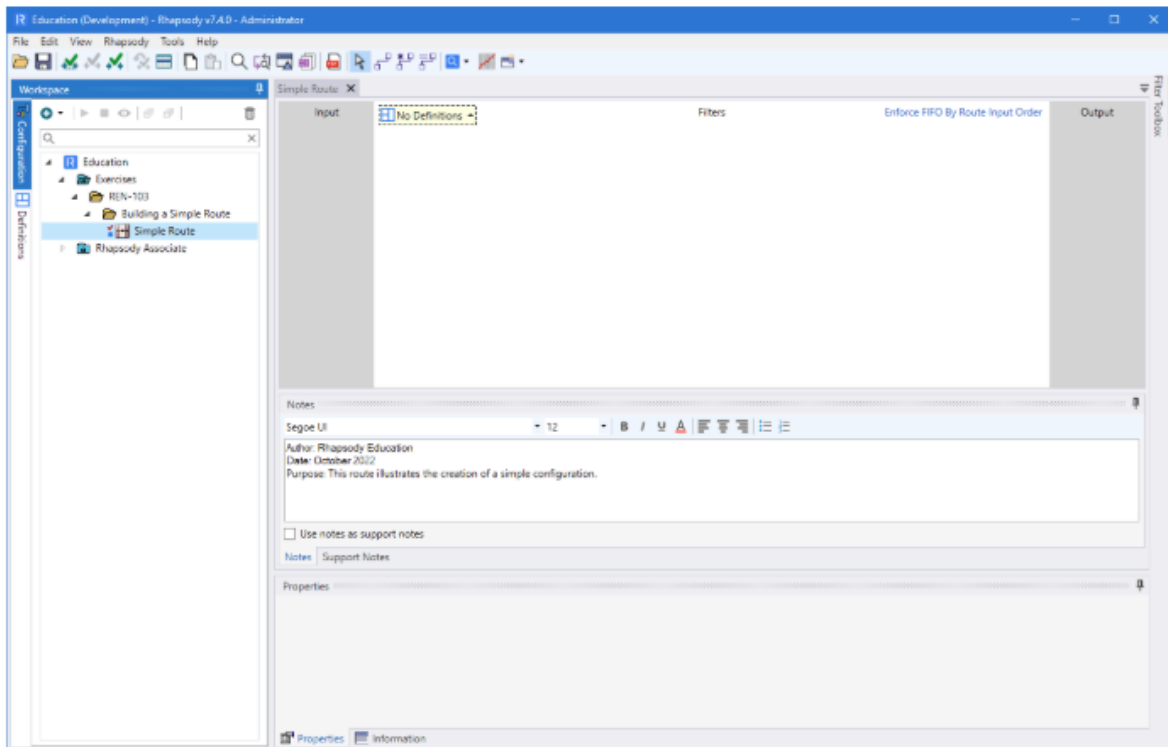
Ensure the **Notes** panel is visible beneath the canvas. If it is not:

1. Open the IDE **View** menu and select **Route Notes**. A tick mark should appear alongside its entry.
2. If the **Notes** entry is ticked but the panel is still not visible, drag the border at the bottom of the canvas upwards until the panel appears.

4. 4

Enter the following details into the **Notes** panel under these headings:

1. **Author**
The name of the administrator responsible for this route's creation and maintenance.
2. **Date**
The date when this route was created.
3. **Purpose**
The purpose of this route along with any explanatory notes that might be required to clarify its configuration.
4. **Modification History** (if appropriate)
A history of the changes, and the reason for them, made to the configuration.



It is Best Practice that the route's purpose be detailed enough that other administrators can quickly understand its configuration and easily maintain your work as required. It is also a good idea to maintain a record of the changes made to the route.

The route is now ready for the addition of communication points and a connector.

Create the communication points

Communication points are the connection between Rhapsody and external systems. We will now create the two communication points required by this exercise. Use the image below the listed steps for guidance.

1. 1

Right-click the working folder for this exercise, (for example, **A. Simple Route**), and select **New Communication Point** from its context menu.

2. 2

Select the **Directory** communication point from the **Commonly Used** list.

3. 3

Enter a meaningful name for the new communication point (we suggest *Message Source*).

4. 4

Click **Create**.

5. 5

On the Configuration tab, change the **Connection Mode** to **Input**. Then edit the **Input Directory** property to specify the directory where Rhapsody will look for incoming messages. The directory you choose should only be used for this exercise. If the directory does not exist, it will be automatically created when the communication point is checked in.

All other properties can be left with their default values for this exercise.

The screenshot shows the 'Message Source Properties' dialog box with the 'Configuration' tab selected. The 'Connection Mode' is set to 'Input'. Below this, a note states 'Properties with an asterisk (*) are required.' A table lists various properties and their values. The 'Input Directory' property is highlighted, showing the path 'C:\Rhapsody\Associate Exercises\REN-103\Input'. Other properties include 'Input Control File Mode' (unchecked), 'Input Control File Extension' (empty), 'Pattern Type' (Wildcards), 'Pattern' (*), 'Exclude Pattern Type' (Wildcards), 'Exclude Pattern' (empty), 'Input Order' (None), 'Comparison Script' (empty), 'Refresh Rate' (5000), and 'Receive Rate' (0). At the bottom, a text box says 'Control files should be expected in input mode'. Buttons for 'Help', 'OK', and 'Cancel' are at the bottom.

Property	Value
Input Directory *	C:\Rhapsody\Associate Exercises\REN-103\Input
Input Control File Mode	☐
Input Control File Extension *	
Pattern Type	Wildcards
Pattern	*
Exclude Pattern Type	Wildcards
Exclude Pattern	
Input Order	None
Comparison Script *	
Refresh Rate	5000
Receive Rate	0

When the engine is installed on a Windows machine, either forward or back slashes can be used in the configuration of a directory path. The recommendation, however, is that back slashes be specified as this is the format that is likely to be familiar to most users.

1. On the **Notes** tab, enter author, date and purpose details in the same way you did for the route above.
2. 7

Select **OK** to create the Directory communication point and add it to the folder.

Create the sink communication point

Repeat and modify the steps above to create a Sink communication point with the name **Discard Messages**. This communication point can only operate in Output mode and has no configurable properties.

Construct the route

Communication points are added to a route using drag-and-drop from the **Workspace**. Remove a communication point from a route by selecting **Delete** from its context menu in the route canvas. Removing a communication point from the **Workspace** also removes it from all routes it is associated with.

1. 1

Drag the Directory and Sink communication points from the folder to the Input and Output sides of the route canvas respectively. Rhapsody will prevent you from placing a communication point configured in Output mode on the Input side of a route. Try it to confirm.

2. 2

Select the **Connector** icon in the IDE toolbar.

3. 3

Hover your mouse pointer over the Message Input Directory communication point. A blue border should appear.

4. 4

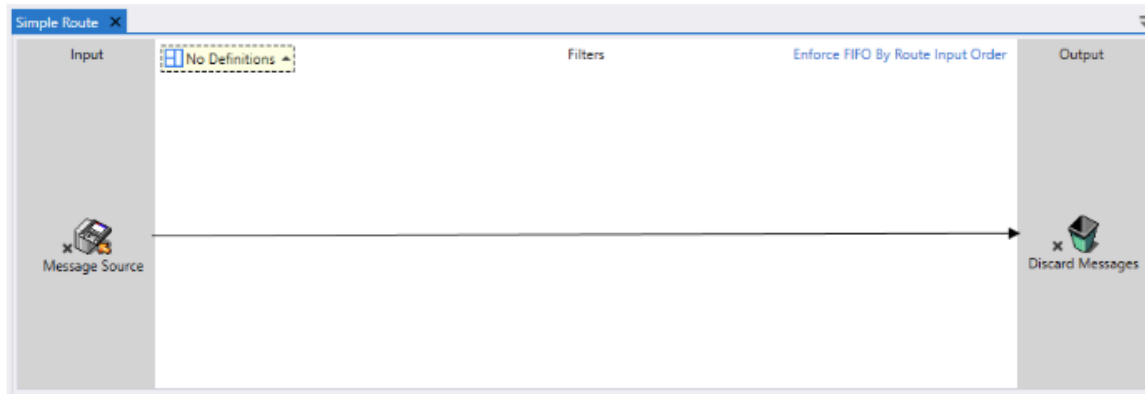
Select and hold down the mouse button, then drag the cursor to the Sink communication point. A blue border will appear.

5. 5

Release the mouse button. The two communication points should now be joined.

What you should see

The completed route should look like the following screenshot:



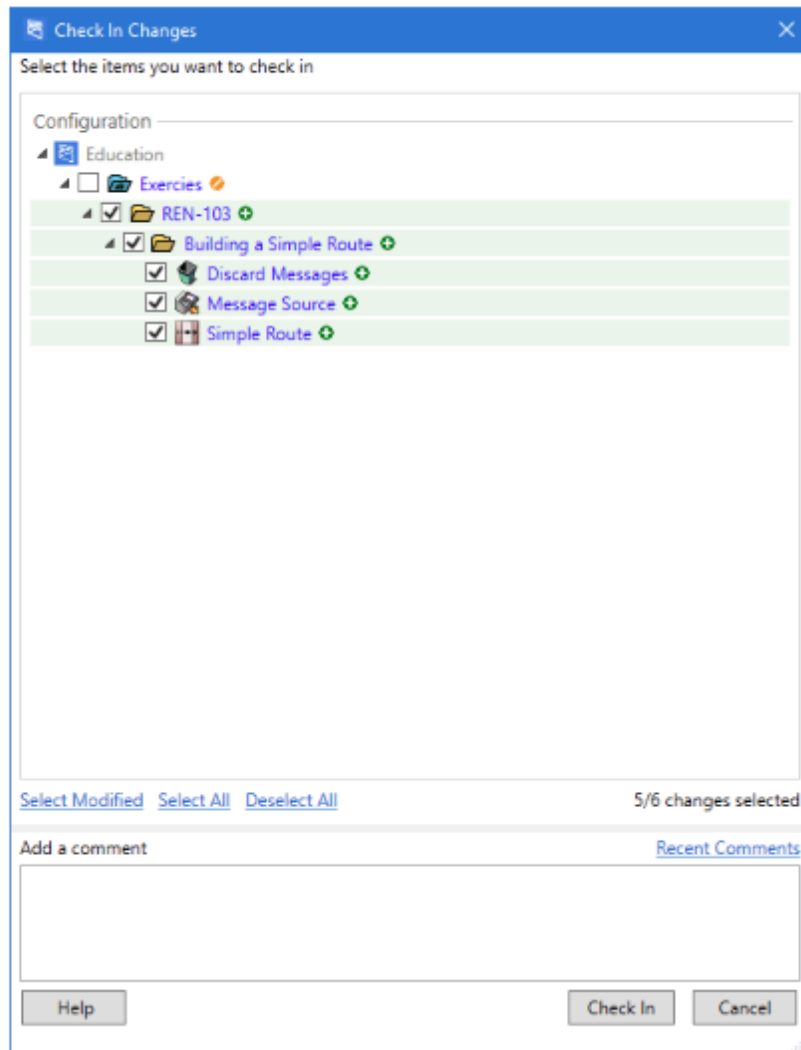
The black crosses alongside the two communication points indicate that their configuration has not been checked in to the Rhapsody engine. We will complete that task next.

Check In and Start Components

Check in components

- 1.

Right-click one of the components you created for this exercise and select **Check In** to display the **Check In Changes** dialog.



2

Select the **Select All** link to select all components. Select **Check In** to complete the action.

Component dependencies

When selecting the above components for check-in, you might notice a Component Dependencies message with the following text:

*"The following components have dependencies due to a communication point being added to a route and must be checked in together. **Message Source, Simple Route.**"*

This means that you cannot check in the communication point without checking in the route at the same time because one is dependent on the other. This will only be true at the time of the route and communication point's creation; after which they can be checked in independently.

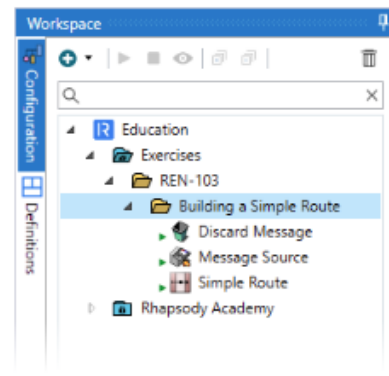
Start components

Communication points and the route(s) they are associated with must be started before message processing can begin.

Right-click each component that needs to be started in the **Workspace** then, from its context menu, select **Start <component>**. Alternatively, right-click their containing folder and choose **Start Components** to start all components simultaneously.

A progress bar might briefly display. The **Information** panel at the bottom of the window will be populated with messages summarizing the success or failure of the action.

Successfully started components are indicated by the presence of a green ► icon next to their entries in the workspace.



Test the Route

We will test the route by copying the test message from the exercise resources to the directory specified during the configuration of the Message Input Directory communication point and observing what happens.

All communication points along with the route must be checked in and running before proceeding with this exercise.

Exercise tasks

1. 1

Copy the *ADTA01.edi* test message from the exercise resources to the communication point's Input directory created above.

2. 2

Refresh the view of the directory's contents. After a few seconds, the file will disappear as it is processed by Rhapsody.

3. 3

Log in to the Management Console. From the **Monitoring** menu, select **Communication Points**.

4. 4

Locate and select the Message Output Sink communication point.
 Note that this screen displays the number of messages processed by each communication point along with the elapsed time since it was last active.

Communication Points (3)

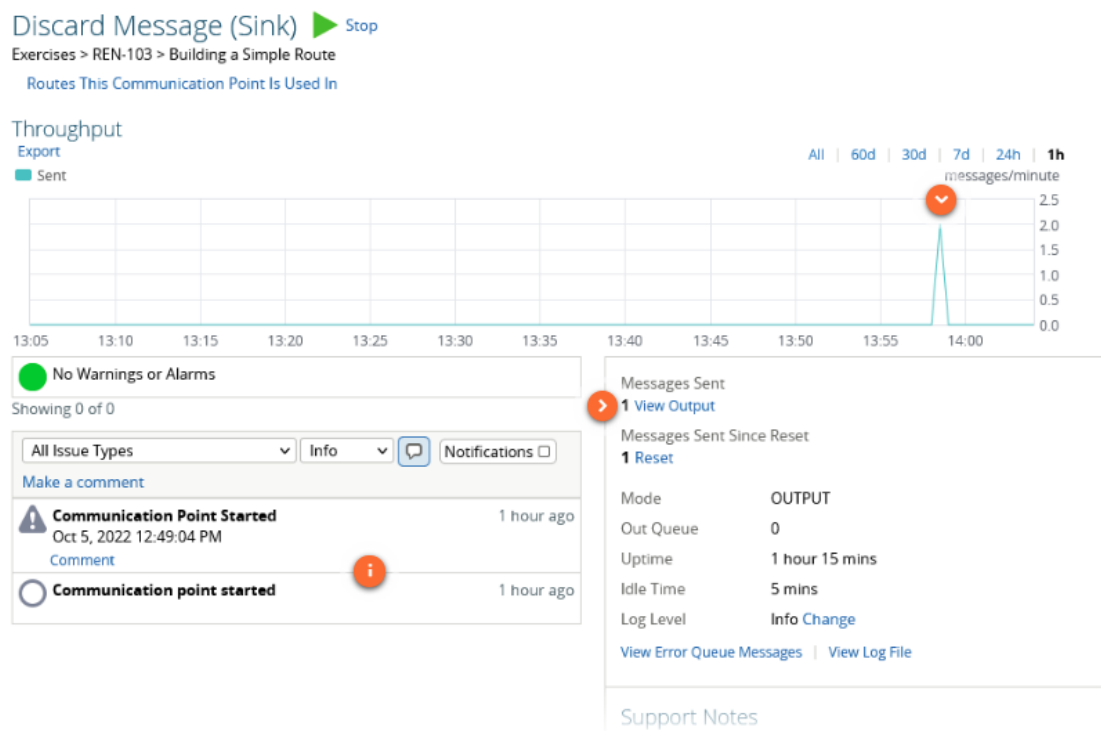
☒ Stopped (4)
 ☒ Running (2)
 ☐ Non-Scheduled (6)

Search by name or type

	COMMUNICATION POINT	TYPE	STATUS	ACTION	CONNECTI	IN QUEUE	OUT QUEU	FAILED QU	RECEIVED	SENT	FAILED	IDLE TIME
☐	Exercises (2)											
☐	REN-103 (2)											
☐	Building a Simple R...											
☐	Discard Message	Sink	Runni...	Stop	1	0	0	0	0	1	0	22 secs
☐	Message Source	Direct...	Runni...	Stop	1	0	0	0	1	0	0	22 secs

5

Select the **View Output** link on the **Details** panel to view the list of messages that have been processed by this communication point.



1. Select the top (most recent) message from the **Message List** if there is more than one.
2. 7

Examine the contents of the **Message Events** tree, **Message Properties** and **Message Body** panels on the **Message View** window.

Model Answer

The following screenshot shows the details for the ADTA01.edi test message after it has been processed by the Simple Route.

Message View 1.768.3

Displaying message 1 of 2 from the result set named 'Discard Message Output Archive (2)' [Back to Result Set](#)

Reinject
Redirect Copy
Redirect Edited Copy
Resend

Message Events !

Always Show Outputs Last refreshed at 14:13

COMPONENT	TIMESTAMP	DESCRIPTION
Message Source	Oct 5, 2022 14:12:51.911	Received by communication point
Simple Route	Oct 5, 2022 14:12:52.053	Placed on route
Discard Message	Oct 5, 2022 14:12:52.054	Delivered to communication point
	Oct 5, 2022 14:12:52.056	Sent by communication point, total pro...

<< < > >>

Message Properties !

Property Name	Property Value
BaseFilename	ADTA01
directory:BodySize	16437
FileCreationTime	1424302386000
FullFilename	ADTA01.edi
InputTime	1664932371911
rhapsody:InputCommunicationPoint	Message Source
rhapsody:InputCommunicationPointId	8
Suffix	.edi

Showing 8 of 8

Message Body !

Tree View
Flat Wire

Show Whitespace ▾ Encoding: US-ASCII
Size: 16437 bytes

```
MISH|^\&|CANNIS|CANNT|CANAB|CANNT|19871010031642||ADT^A01^ADT_A45|10215605xgfd|D^Not|2.4^23^14|6789||AL|AL|VCT||19|2.3\r
EVN|509|20021017031731|19891027041955|02|54^EVERLY^L^PREE ^^Jr^Eng^DBA^ST123^CANNT^P^3^M10^NH^CANNU^^13^19971025155749&19871010111044
PID|1654|S^5^M10^AUSHIC^DS^USDC^19991010^19931010|2^2^ISO^CANNB^DN^CANQC^19981020^19821014~54^1^ISO^CANNT^UPIN^USDC^19981030^199211
PDI|C^M^CB|I|^L^24^5^ISO^AUSHIC^DI^CANON~^L^4^6^M11^CANAB^ET^CAINT~^L^2^3^M10^AUDOVA^LR^CANNF|54^WILLIS^MALISSA ^^Sr^DR^BBA^ST123^CAI
ROL|1^AUSHIC|AD|AD|24^STROUD^BERNE ^^Sr^Prof^BSL^ST123^CANNU^S^4^M11^RR^CANVT^^22^19901110202350&19941010023710^G|19981026103540|199:
ROL|11^CANMS|LI|PP|1^BOSWELL^SHERYL ^^Sr^DR^MDI^ST123^CANVT^L^6^M10^DS^CANVT^^5^19971110024512&19871010175154^G|19921010022615|19811
ROL|5^CANQC|UC|RP|5^ODLE^CLEVELAND ^^III^Dr^DBA^ST23^CANAB^M^2^M11^VS^CANON^^9^19911110091341&19991014091250^G|19901010183319|199110:
NKI|1654|ROMINES^QUEENIE ^^III^Eng^MD^C^^13^19851010174850&19891023003156~YOUNGST^EAD^FARICA ^^II^DR^MED^B^^8^19921011094736&20021010
NKI|4567|HORTON^CLODIA ^^I^Eng^BA^L^^17^19811016051025&19871015063310~FALEY^JACQUeline ^^Jr^Eng^DO^R^^13^19941010161033&1991101015: \n
```

<
>

Download

Message events tree

This panel lists the various route components that the message has passed through as it was processed by the engine. Select each one to see if the message's associated properties change as processing progresses. No changes should be present in this example, but in more complex routes, checking this information is one method for confirming correct processing or locating and diagnosing errors.

The navigation icons at the bottom of this panel can be used to select the next or previous message from the route archive, or to move up or down through the listed message events.

Message properties

This panel lists the message's associated properties. The FullFilename property, for example, will match the full name of the test message used in this exercise.

Message body

This panel displays the content of the message at the various stages of processing. It will be identical to the content of the test message itself, which you can view in a text editor such as Notepad or Notepad++. Even though the test message is in HL7 format, it can only be viewed in the **Flat Wire** view (as shown) because a matching message definition has not been associated with the route to define the message's structure.

Exercise - Message Properties

In the previous exercise, we created a route but did not attempt to change or control the messages passing through it. Messages were accepted by the Input communication point, accepted by the route, then passed unchanged to the Output communication point for discard.

The purpose of the following exercise is to investigate the effect on the properties (metadata) associated with messages at each stage of processing after adding a Message Definition and Filter to a route. These properties will be used in a later exercise to selectively route messages.

This exercise is in two parts:

1. Add a message definition to a Rhapsody configuration.
2. Modify the route by adding a filter, then observe how this changes message processing.

3. Resources

4. This exercise uses the messages and message definitions in the *.\Associate Files and Definitions\103 - Building a Simple Route* folder in the resources Zip file available at the start of the course.

5. Part 1: Add a Message Definition

6. A Message Definition is a file that describes the structure and format of a message and can be used to validate its content. This exercise uses the **Academy-HL7v2.4-ADT.s3d** message definition in the exercise resources.

7. Load a message definition

8. Complete the following steps to add a message definition to the engine. Use the image below the listed steps for guidance.
9. Select the **Definitions** tab in the workspace.

10.2

Right click the locker containing the folders, routes, and communication points you created for your Associate course exercises.

11.3

From the dialog box that appears, select **Add Definition > Add Existing Definitions**.

12.4

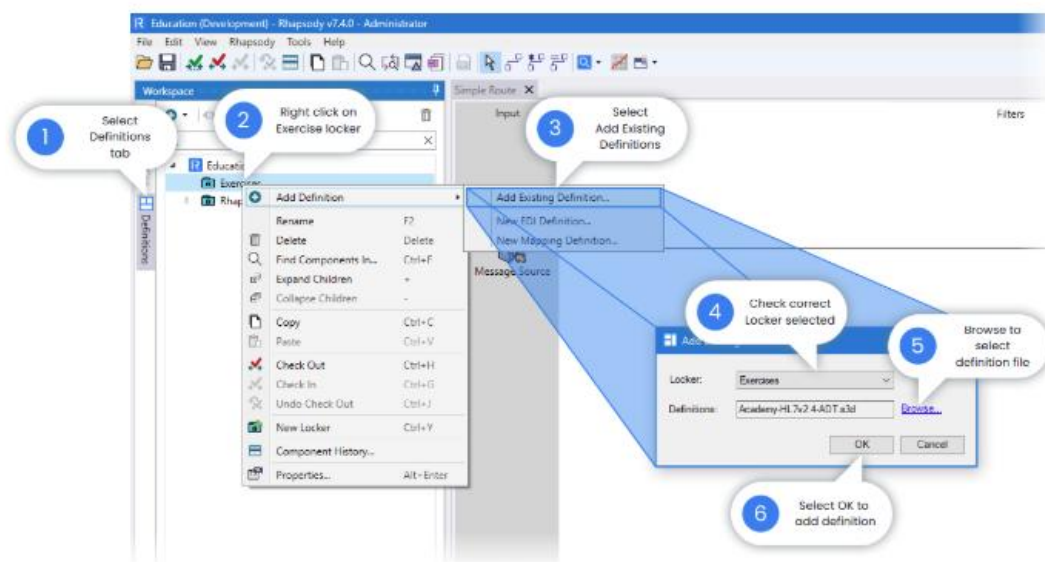
Check that the **Locker** selected is your exercise locker. If the locker name is incorrect, change it to the expected locker.

13.5

Select the **Browse** link and navigate to the `Academy-HL7v2.4-ADT.s3d` message definition. Select the file, then click **Open**.

14.6

Select **OK** and close the **Add Existing Definition** window.



After the definition has been checked in, you can use it in the configuration of both routes and filters. A definition can be used many times within the same solution.

Associate the message definition with a route

We will now add the definition that we added above to a route. Use the image below the listed steps for guidance.

1. 1

View the route created for *Exercise - Building a Simple Route*

Exercise - Building a Simple Route

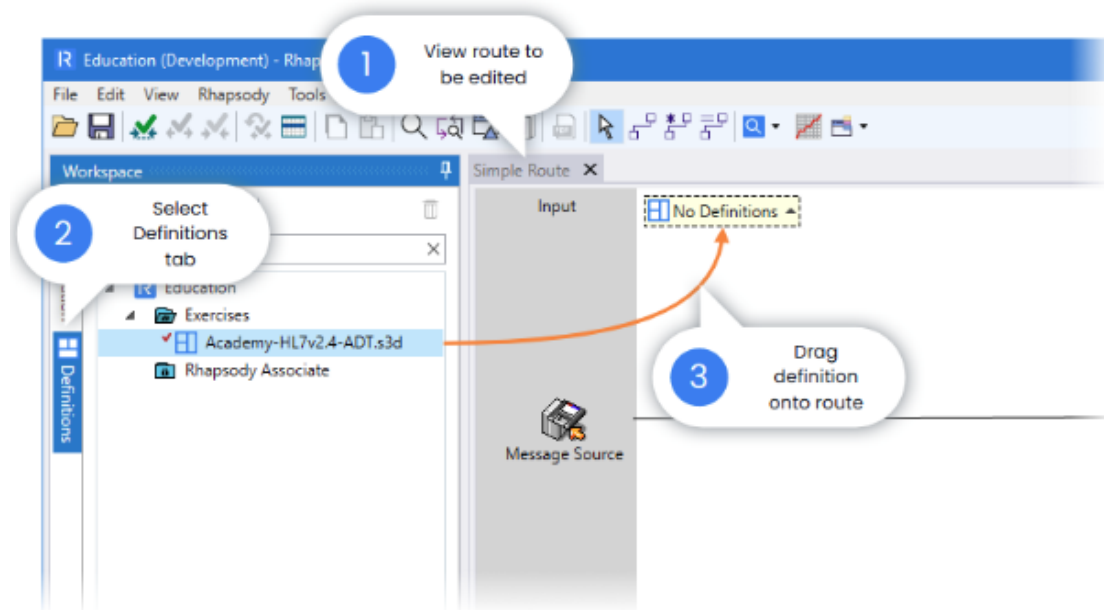
[Go to exercise](#)

1. 2

Select the **Definitions** tab on the **Workspace**. Ensure the new definition is present and located in the correct locker.

2. 3

Select the new message definition, then drag its icon to the **Message Definitions** panel on the route editing canvas.



Associating a message definition with a route automatically checks out that route. The route must be checked in before you can see the effect of adding the definition.

Investigating this will be the subject of a later exercise.

When adding a message definition to a route, both the definition and the route must be in the same locker. If the definition is in a different locker, it must be copied to the locker holding the route. Select the **Copy to Another Locker** option, then select the locker to copy to. The copied definition must be checked in before it can be used in the route's configuration.

Testing

Make sure the route and its components are checked in and started. Complete the following steps:

1. 1

Place a copy of the ADTA01.edi test message into the Input directory for the **Message Source** communication point.

2. 2

Open the Management Console.

3. 3

Locate and select the **Discard Messages** sink communication point associated with this exercise.

4. 4

On the communication point's **Details** page, select the **View Output Archive** link.

5. 5

Select the most recent message to open its **Message View**.

6. 6

In the **Message Events** panel, select the **Discard Messages** communication point.

View the contents of the **Message Properties** panel at this stage of the message's processing.

Model Answer

The same six properties are associated with both this message and the messages processed by this route (without the message definition) in the previous exercise. The addition of a message definition alone to a route has had no effect on the messages following their processing.

Message Properties ▾

Property Name	Property Value
BaseFilename	ADTA01
directory:BodySize	16437
FileCreationTime	1562073609243
FullFilename	ADTA01.edi
InputTime	1562073612598
rhapsody:InputCommunicationPoint	Message Source
rhapsody:InputCommunicationPointId	49
Suffix	.edi

Showing 8 of 8

The next part of this exercise examines the effect of adding a filter to the route.

Part 2: Add a Filter

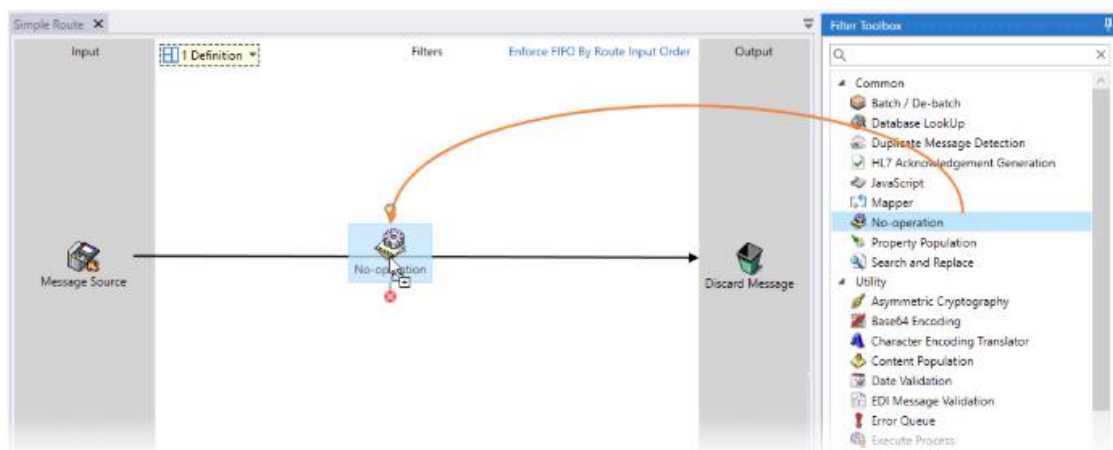
To add a filter to a route, drag its icon from the **Filter Toolbox** onto a route connector. Filters provide processing logic to messages as they move through the route. For example, the action of a filter can change the structure and/or content of a message depending on specific business needs. The **No-Operation** filter is often used as the starting point for branching pathways but does not itself apply any changes to messages during processing.

Adding the **No-operation** filter to the simple route forces all messages to be parsed by the engine. This means that information regarding the message's type and structure will be extracted and, in this case, will result in message properties being implicitly added to the message.

Complete the following steps to add a filter to the simple route:

1. 1

Select the **No-operation** filter and drag it onto the connector between the Input and Output communication points. When the filter is correctly placed over the connector, it will display as a bold line.



2

Release the mouse button to divide the connector at that point and insert the filter.



The filter initially displays with a grey cross to indicate that the changed configuration has not been validated by the engine. No configuration is required with a No-operation filter. All you need to do is check in the route. Filters are checked in along with the route they are part of.

3

Right-click and change the name of the **No-operation** filter to better reflect its function in the route, for example, *Pass Through* (as messages pass through this filter without being changed).

4

Check in the route to save the changes (either right-click in the route editing canvas to access the context menu, or right-click the route's icon in the **Workspace**).

Testing

Make sure the route and its components are checked in and started, before completing the following steps.

1. 1

Place a copy of the ADTA01.edi test message into the Input directory for the **Message Source** communication point.

2. 2

Open the Management Console.

3. 3

Locate and select the **Discard Messages** communication point associated with this exercise.

4. 4

On the communication point's **Details** page, click the **View Output Archive** link.

5. 5

Select the most recent messages to open its **Message View**.

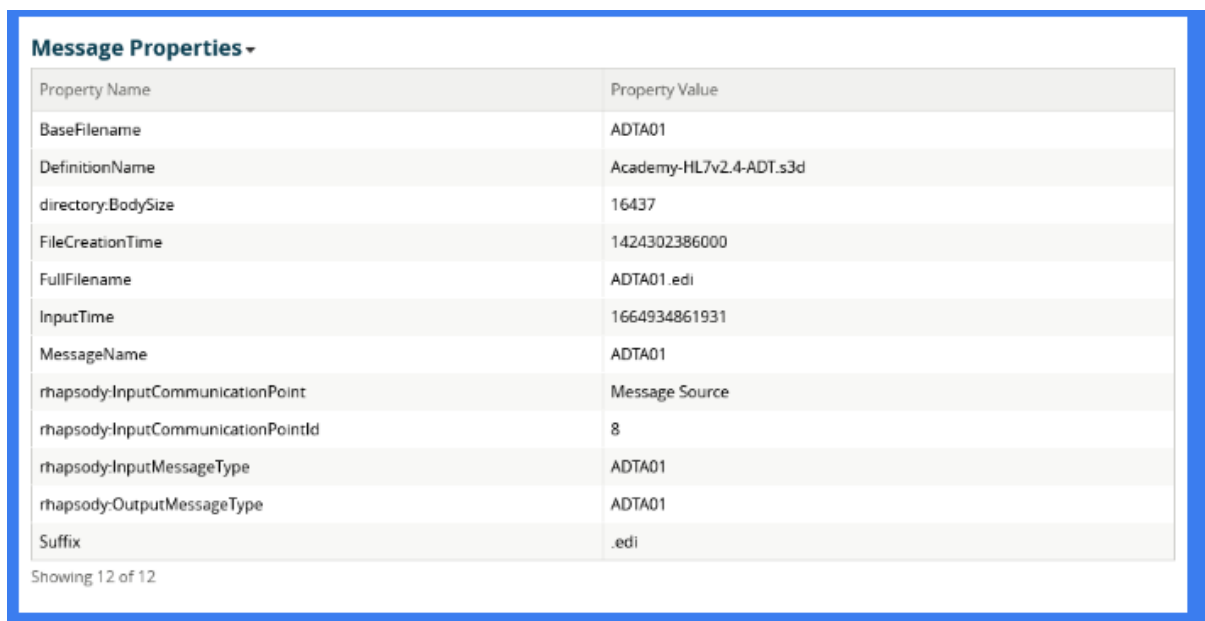
6. 6

In the **Message Events** panel, select the **Discard Messages** communication point.

View the contents of the **Message Properties** panel at this stage of the message's processing.

Model Answer

Four additional properties have been associated with the test message following its parsing by the route's **No-operation** filter. The following screenshot shows these four new properties:



The screenshot shows a 'Message Properties' panel with a table of 12 properties. The table has two columns: 'Property Name' and 'Property Value'. The properties listed are: BaseFilename (ADTA01), DefinitionName (Academy-HL7v2.4-ADT.s3d), directory:BodySize (16437), FileCreationTime (1424302386000), FullFilename (ADTA01.edi), InputTime (1664934861931), MessageName (ADTA01), rhapsody:InputCommunicationPoint (Message Source), rhapsody:InputCommunicationPointId (8), rhapsody:InputMessageType (ADTA01), rhapsody:OutputMessageType (ADTA01), and Suffix (.edi). At the bottom, it says 'Showing 12 of 12'.

Property Name	Property Value
BaseFilename	ADTA01
DefinitionName	Academy-HL7v2.4-ADT.s3d
directory:BodySize	16437
FileCreationTime	1424302386000
FullFilename	ADTA01.edi
InputTime	1664934861931
MessageName	ADTA01
rhapsody:InputCommunicationPoint	Message Source
rhapsody:InputCommunicationPointId	8
rhapsody:InputMessageType	ADTA01
rhapsody:OutputMessageType	ADTA01
Suffix	.edi

Showing 12 of 12

The four new properties are:

- **DefinitionName**
The name of the definition applied to parse the message.
- **MessageName**
The HL7 message type from the message.
- **rhapsody:InputMessageType**
The HL7 message type for the input message.
- **rhapsody:OutputMessageType**
The HL7 message type for the output message.

Explanation

Rhapsody does not attempt to interpret a message unless required to. In other words, Rhapsody will not consider the message's structure or content when it is

processed. We observed this behavior in the first exercise of this module when we were unable to view the tree view for a message processed by a route that did not utilize the **No-operation** filter. Rhapsody does this for performance reasons - it is faster to move a message without looking inside it first.

Even when a message definition is associated with a route, Rhapsody will only interpret the message when it needs to understand the message structure. Only when a filter is added to the route will Rhapsody parse the message using the route definition. Parsing also causes Rhapsody to add the **DefinitionName**, **MessageName**, **rhapsody:InputMessageType** and **rhapsody:OutputMessageType** properties to the message - which is what we see in this exercise.

Conditional Connectors

o control a message's path, conditions can be applied to the connectors leaving a route. Message can be filtered by:

- Message type
- Message property
- Field within a message
- Message body

No-Match and Error Connectors

Special connectors appear above and below a filter to route messages that can't pass along a normal connector leaving the filter.



No-Match connector

Messages that do not meet the conditions set on any of the normal connectors are sent to this connector. Do not use the No-Match connector if there are not any conditions on the normal connectors leaving the filter.

Error connector

Messages that trigger an error during processing on the filter are sent to this connector.

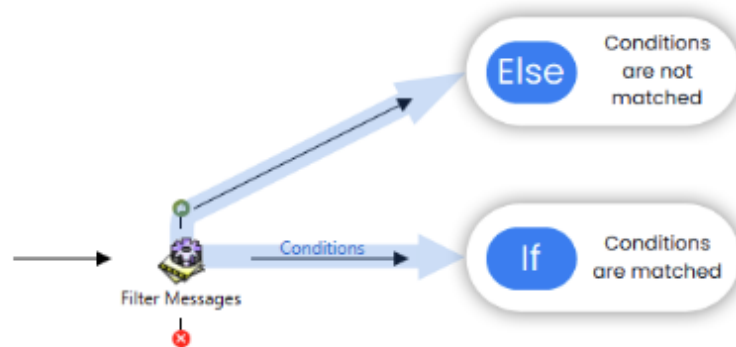
Processing Logic

A filter will first attempt to pass the message to each connector. If the message does not match the conditions of any connector, it will be passed to the No-Match

connector. If this connector is not connected, any messages that would be sent to it will be sent to the Error connector instead. Also, if an error occurs while processing the message through the filter, it will be sent to the Error connector (or Error Queue if the Error connector is not connected).

The processing logic for the use of the Conditional and No-Match connectors is:

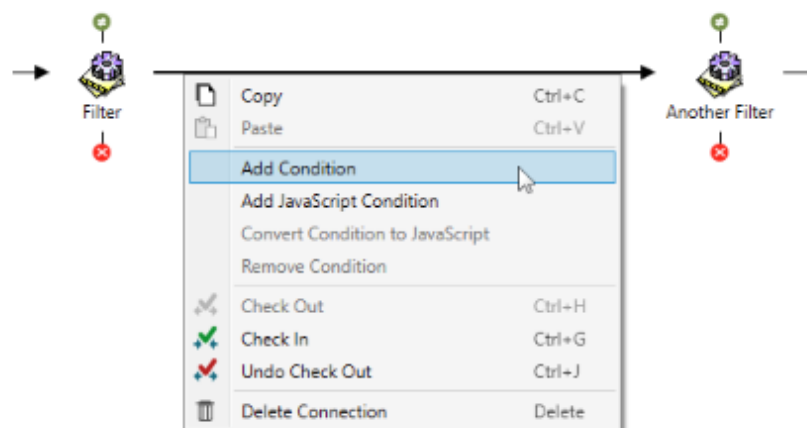
- If the message matches the criteria set on the conditional connector, it is passed along that connector.
- Else, if the message does not match any conditions, it is passed to the No-Match connector.



Create a Condition on a Connector

There are two ways to add a conditional connector. Either way creates a conditional connector that is ready to be configured.

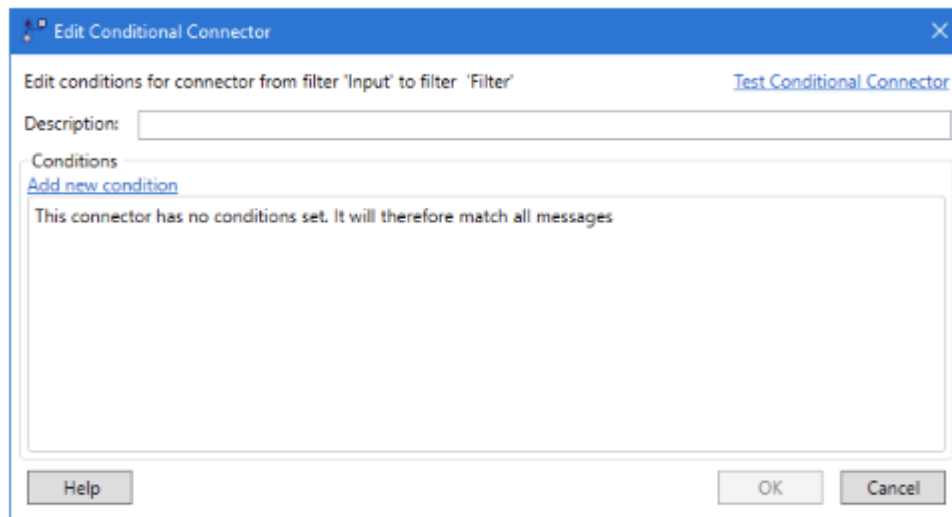
1. Select the **Conditional Connector** icon on the toolbar and create the new connection the same way you would for any other connection.
2. Right-click an existing connection and select the **Add Condition** link from the context menu.



A conditional connector can only originate from a filter and not from a communication point. This is because communication points do not have an associated No-Match connector, so messages not meeting the criteria can never be properly routed. Rhapsody will not allow conditions to be added to a connector that originates from a communication point.

Configure a conditional connector

Select the text of a conditional connector to open the **Edit Conditional Connector** window.



From this window you can perform three tasks:

1. Add a description that describes the logic of the connector.

✎ You should always add a brief description to a conditional connector. The description should create an "at-a-glance" summary of what messages will flow through the connection. In the absence of a description, an auto-generated summary will be displayed on the connection; this is often long and complex.

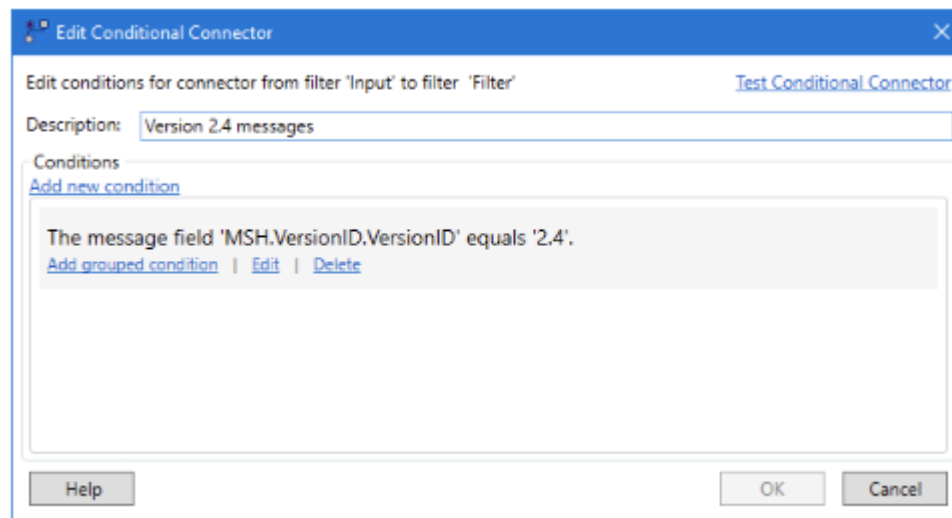
For example, by default a condition that filters messages by message type might look like this: **(ADTA36)(ADTA47)(ADTA04)(ADTA48)(ADTA27)(ADTA08)**, which is too long to view in the Route Editor. In this case, a label of **ADT** would likely be sufficient (assuming the set represented all ADT messages the condition was expected to filter).

1. 2

Add or edit conditions for the connector. A conditional connector may have any number of conditions. These can be grouped with a combination of **And** (all conditions must be true) or **Or** (only one of the conditions needs to be true) logic.

2. 3

Test the conditional connector. The exercise that follows this lesson demonstrates testing a conditional connector.



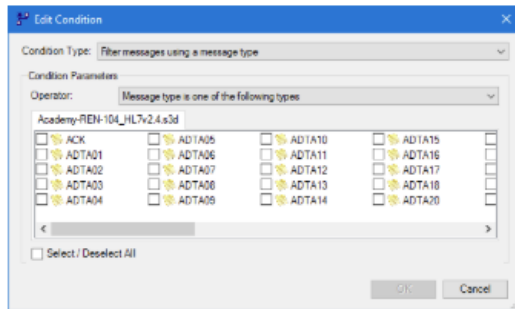
Add/edit conditions

To add a condition to a connector, select the **Add new condition** link. To edit an existing condition, select the **Edit** link. In both cases the **Edit Condition** dialog opens.

When creating a new condition, you should first select the appropriate **Condition Type**. The behavior of the **Condition Parameters** depends on the selected condition type.

Filter messages using a message type

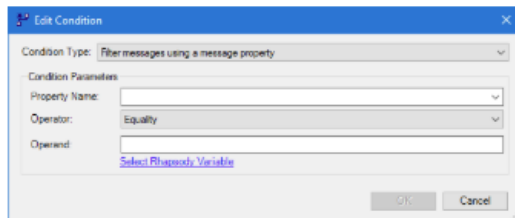
Filter messages using a message type



This condition type lets you restrict messages according to their message type and can only be used when a message definition is included on the route.

Condition Type: Filter messages using a message type.

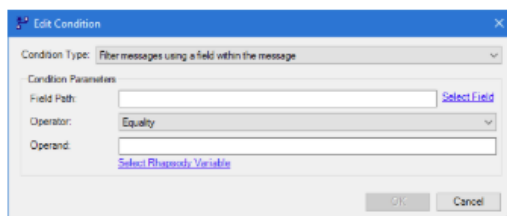
Filter messages using a message property



This condition type lets you inspect properties associated with a message and filter messages based on their content.

Condition Type: Filter messages using a message property.

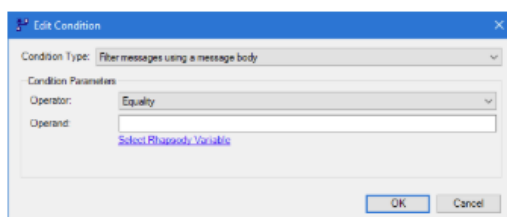
Filter messages using a field within the message



This condition type lets you inspect fields in a message and filter messages based on their content. This condition type can only be used when a message definition is included on the route.

Condition Type: Filter messages using a field within the message.

Filter messages using a message body



This condition type lets you filter messages based on the content of the message body.

Condition Type: Filter messages using a message body.

Operators

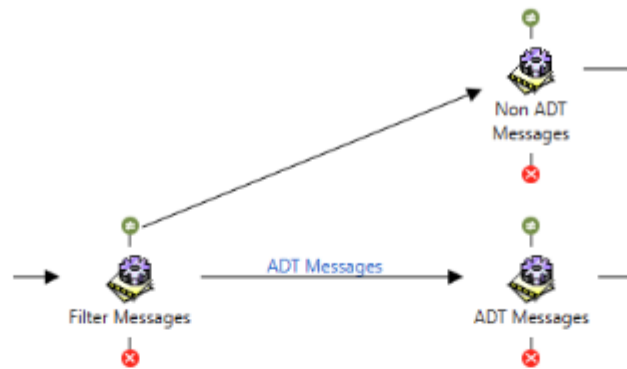
Rhapsody supports many different operators to compare the part of the message being tested with the operand. During the exercise that follows this lesson you might want to review the operators that are available and consider how you could use each of them.

Best Practices

The following route layouts show how rules-based conditional connectors can be included in a route. For example, multiple conditional connectors, or multiple conditional connectors and one or more unrestricted connectors, can be configured to leave a filter.

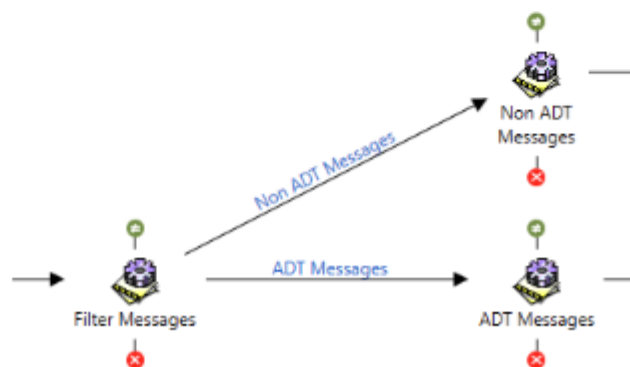
Recommended layout

In the following example, ADT messages are routed to the ADT Messages filter. All other messages are routed via the No-Match connector to the Non ADT Messages filter.



Not recommended – conditional connector layout

In the following example, ADT messages are routed to the ADT Messages filter, similar to the recommended layout. However, we have defined a second conditional connector to send all other messages to the Non ADT Messages filter. In this configuration, a message that does not match either conditional connector is sent to the Error queue (as there is no connection from the No Match connector).

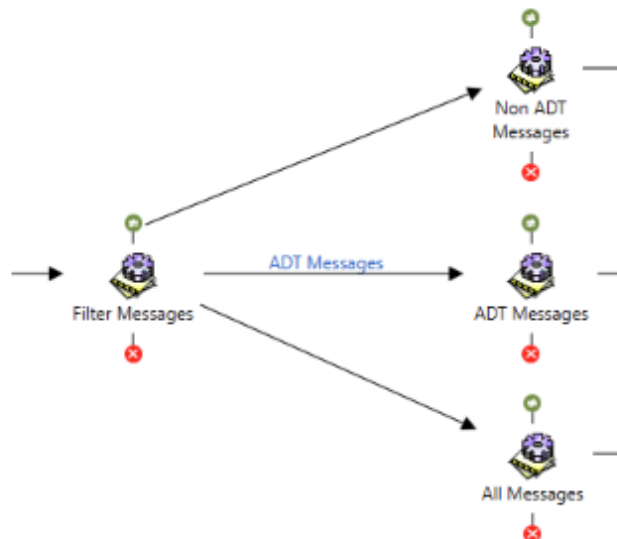


The recommended layout is preferred for the following reasons:

- Route configuration is simplified; only one conditional connector needs to be set up.
- Route interpretation is easier as only one rule needs to be checked to determine where messages will go.
- Route performance is improved as the engine is only required to evaluate message structures once.
- There is no risk of a fault in the rules sending messages to the Error queue (as the No Match connector has not been configured as a destination for messages that do not match any conditional connectors).

Not recommended - no conditions specified

In the following example, there is an unrestricted connection leaving the Filter Messages filter. As a result, the No-Match connector would never be used, and no messages would ever be sent to the Non ADT Messages filter.



This configuration is not recommended because it includes an unnecessary connector which could be confusing to other administrators.

Exercise - Decision Making

This exercise creates a conditional connector to selectively route messages based on their structure and/or content. The test messages in this exercise are based on the HL7 V2.4 standard and include ADT (Admit, Discharge, Transfer), ORM (General Order) and ORU (Observation or Result) message types.

We will configure a conditional connector to send ADT messages to an ADT Sink communication point. All other message types go to a non-ADT Sink communication point.

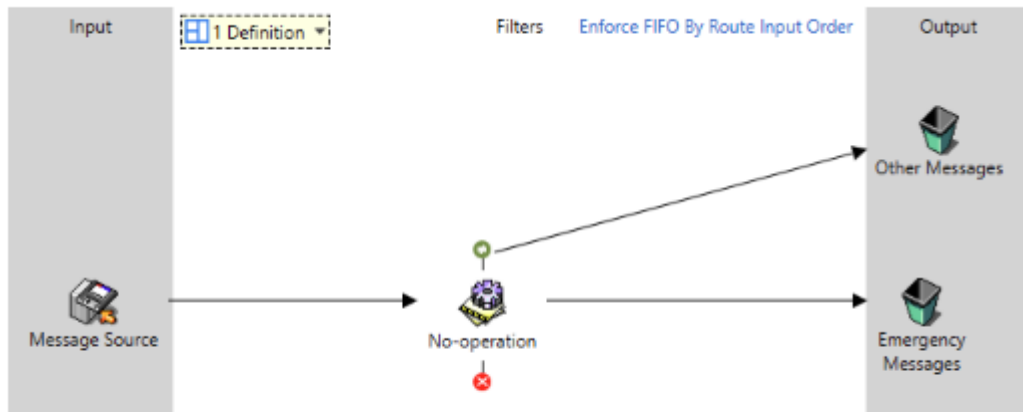
Resources

This exercise uses the messages and message definitions in the . \Associate Files and Definitions\103 - Building a Simple Route folder in the resources Zip file available at the start of the course.

Add a conditional connector

Begin by constructing the Decision Making route shown below. Follow the steps in the previous module to add the Academy-HL7v2.4-ADT.s3d message definition to the route.

Follow the steps for **Associate the message definition with a route** in the previous exercise to add the Academy-HL7v2.4-ADT.s3d message definition to the route.



To configure the conditional connector:

To configure the conditional connector:

- 1 Click the connector that connects the No-operation filter to the ADT Sink communication point. The connector will be change to bold to confirm the selection.
- 2 Right-click on the bolded connector and select **Add Condition**.
- 3 Click the **Add new condition** link.
- 4 On the **Edit Condition** window for **Condition Type**, select **Filter messages using a field within the message**.
- 5 Select the **Select Field** link.

Edit Condition

Condition Type: **Filter messages using a field within the message**

Condition Parameters

Field Path: [Select Field](#)

Operator: **Equality**

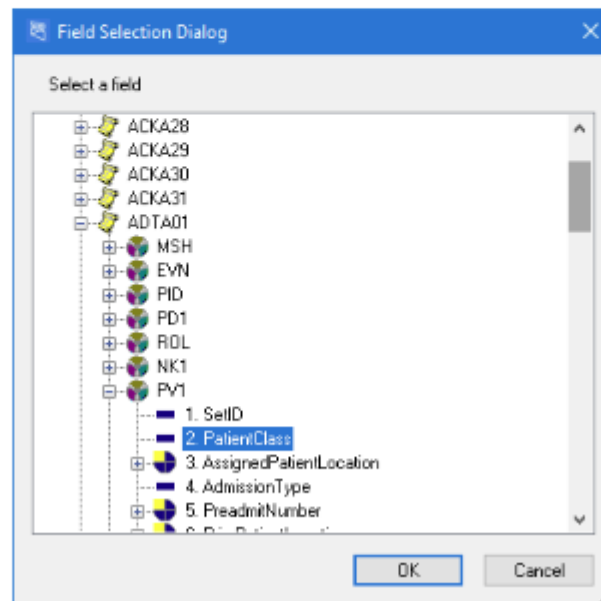
Operand: [Select Rhapsody Variable](#)

OK **Cancel**

6 Expand the **ADTA01** message followed by its **PV1** segment.

7 Select the **2. PatientClass** field.

8 Select **OK**.



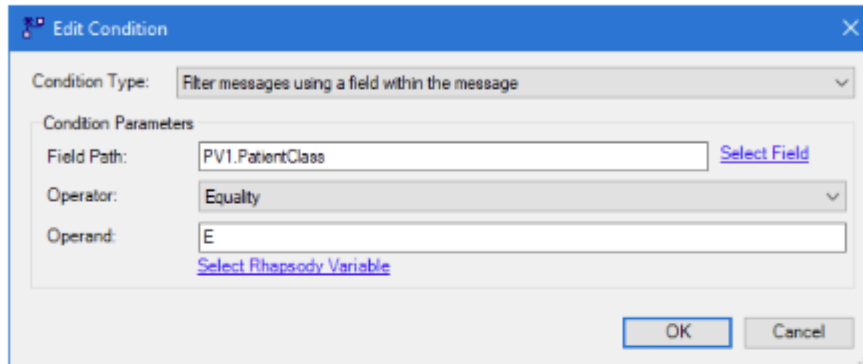
Because the PV1 segment is common to all HL7 ADT messages in the definition. Only a representative example needs to be selected; in this case we used the ADTA01 message type.

9

Ensure the Field Path is `PV1.PatientClass` and the Operator is **Equality**.

10

Type *E* as the condition's Operand



The 'Edit Condition' dialog box is shown with the following settings:

- Condition Type: Filter messages using a field within the message
- Field Path: `PV1.PatientClass` (with a [Select Field](#) link)
- Operator: Equality
- Operand: `E` (with a [Select Rhapsody Variable](#) link)

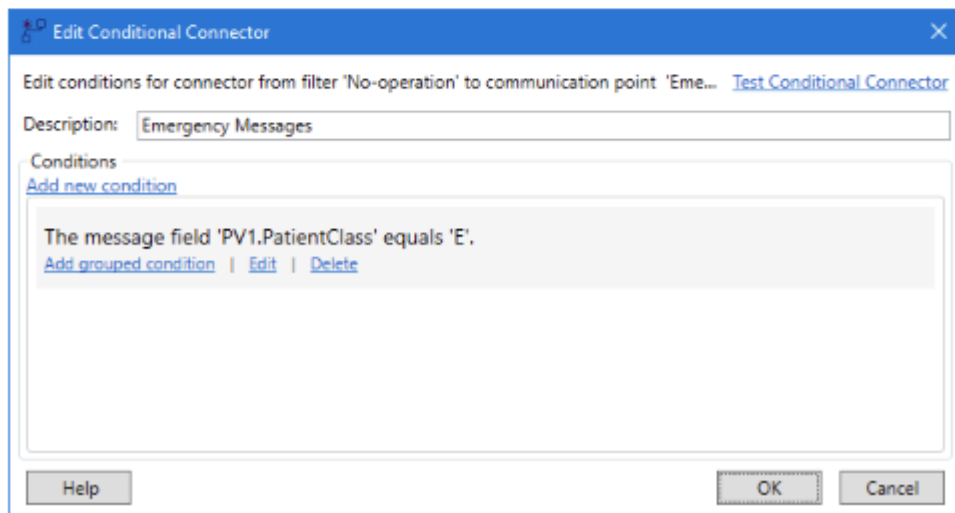
Buttons at the bottom: OK, Cancel.

11

Select **OK**.

12

Reopen the conditional connector's configuration window and edit its description to *Emergency Messages*.



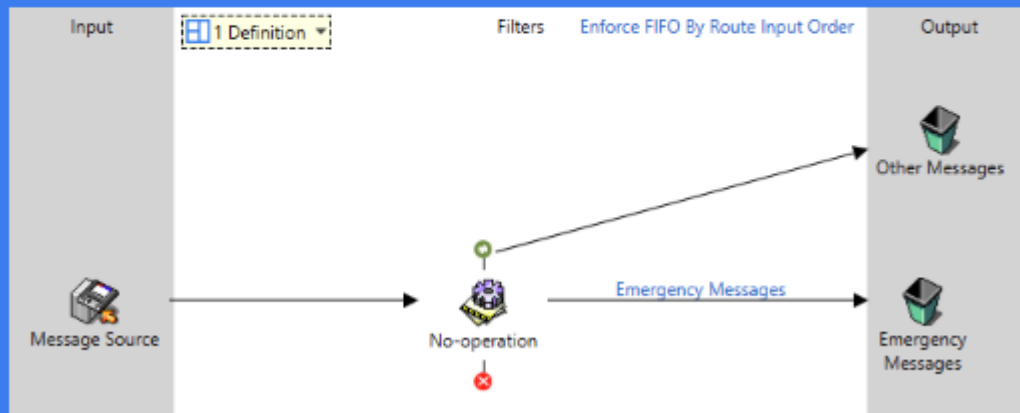
The 'Edit Conditional Connector' dialog box is shown with the following settings:

- Description: `Emergency Messages`
- Conditions: [Add new condition](#)
 - The message field 'PV1.PatientClass' equals 'E'.
[Add grouped condition](#) | [Edit](#) | [Delete](#)

Buttons at the bottom: Help, OK, Cancel.

Model Answer

The completed configuration should look like this:



Testing the Route

We will now examine what happens when we process some test messages through the configuration.

- 1 Right-click each component to start the route and its three communication points.
- 2 Place a copy of the following three test messages into the route's Input directory. Watch until the files have been picked up and placed into the route for processing.
 - *ADTA01.edi*
 - *ADTA03-1.edi*
 - *ADTA03-2.edi*
- 3 Open the Management Console.
- 4 From the **Monitoring** menu, select **Communication Points**.
- 5 Locate the **Emergency Messages** and the **Other Messages** communication points.
- 6 Click the **View Output Archive** link for each communication point, followed by its most recently processed message.
- 7 Select the **Output** communication point in the **Message Events** panel.
- 8 In the **Message Properties** panel, locate the full filename of the test message and confirm that the ADTA01 and ADTA03 test messages are going to different destinations. This means the conditional connector is working as expected.

Model Answer

The ADTA01 test message is delivered to the ADT Sink communication point while the two non-ADT messages are routed to the non ADT Sink communication point. Messages must be viewed at the output communication point of the **Message Events** tree to verify they have been correctly separated.

The following screenshot shows the ADTA01 message at the **Emergency Messages** communication point.

Message Events ▾

Always Show Outputs ☐

Last refreshed at 11:55

COMPONENT	TIMESTAMP	DESCRIPTION
Message Source	9.294	Received by communication point
Conditional Connectors	9.300	Placed on route
	9.302	Property extraction performed
No-operation	Oct 12, 2022 11:55:19.302	Filtering succeeded; took 0.003 ms
Emergency Messages	Oct 12, 2022 11:55:19.309	Delivered to communication point
	Oct 12, 2022 11:55:19.314	Sent by communication point, total proc...

Message Properties ▾

Property Name	Property Value
BaseFilename	ADTA01
DefinitionName	.4-ADT.s3d
directory:BodySize	
FileCreationTime	1665528726000
FullFilename	ADTA01.edi
InputTime	1665528919294
MessageName	ADTA01
rhapsody:InputCommunicationPoint	Message Source
rhapsody:InputCommunicationPointId	125
rhapsody:InputMessageType	ADTA01
Show More	

Showing 10 of 12

The two ADTA03 test messages will be routed to the non Other Messages output communication point. Their corresponding message details screenshots will be similar to the one above.

Explanation

Because an HL7 V2.4 message definition is associated with the route and the test messages were all based on this version, Rhapsody can locate specific content within each message and use it to intelligently route the messages. In this example, the value contained in the PV1.PatientClass field is determined and used to send each message to either the Emergency or Other Messages communication points.

Conditional Connector Testing

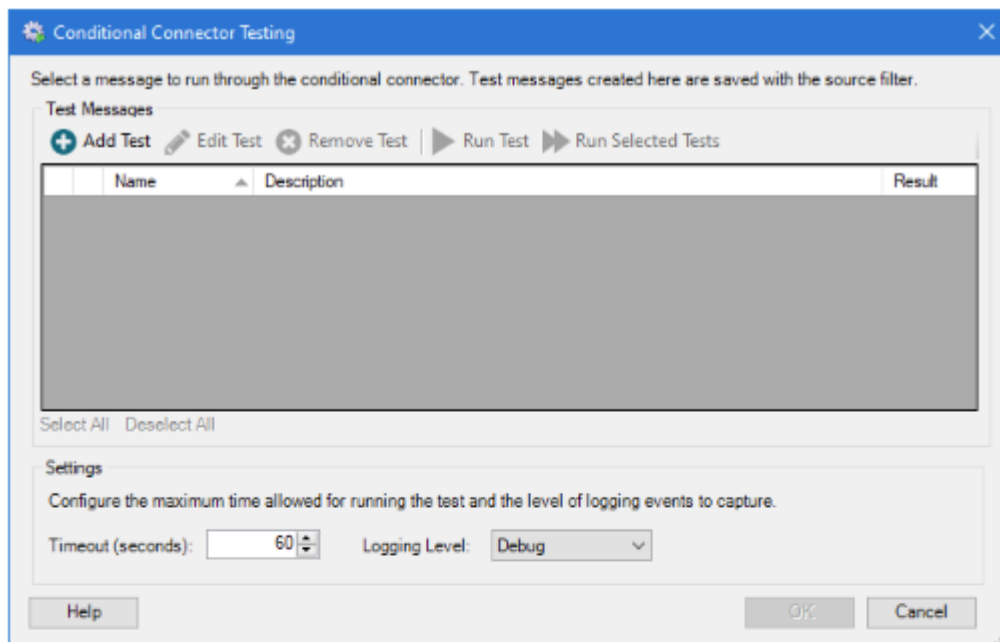
A conditional connector can be tested without starting the route and communication points it is associated with. However, they should be checked in. Complete the following steps to test a conditional connector:

- 1

Select the conditional connector's name in the route to open the **Conditional Connector Testing** window.

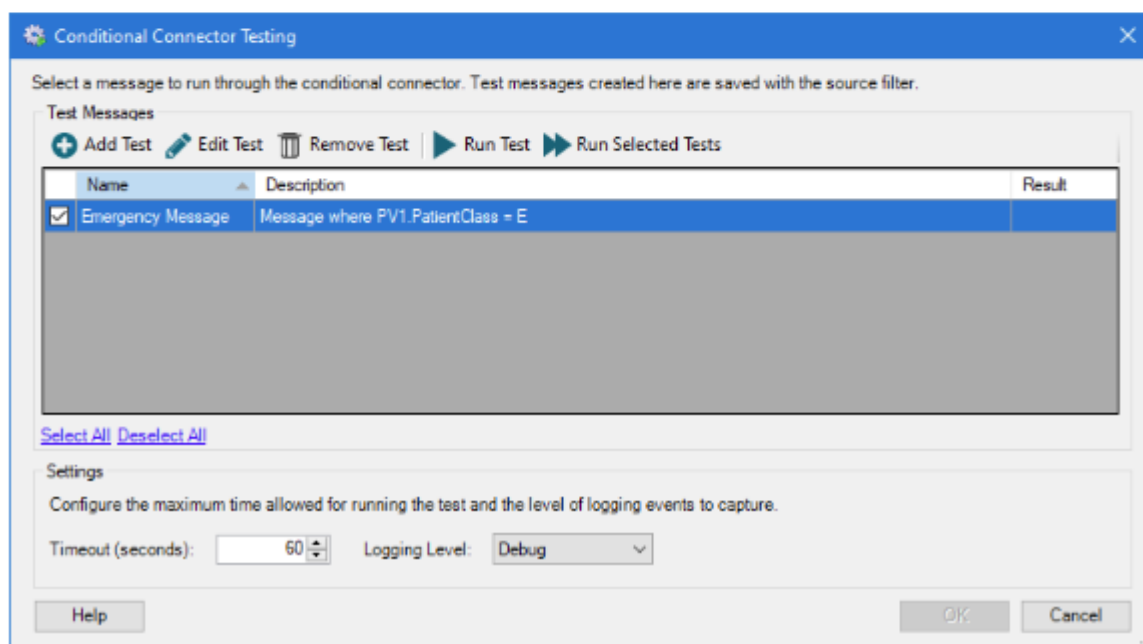
- 2

Click the **Test Conditional Connector** link, then select **Add Test**.



3 Provide a name and description for the test message.

4 Click the **Load Message** link and navigate to the exercise resources and select the **ADTA01.edi** test message. To confirm you have loaded the correct message, locate its message type in the top row; it should be **ADT^A01**.



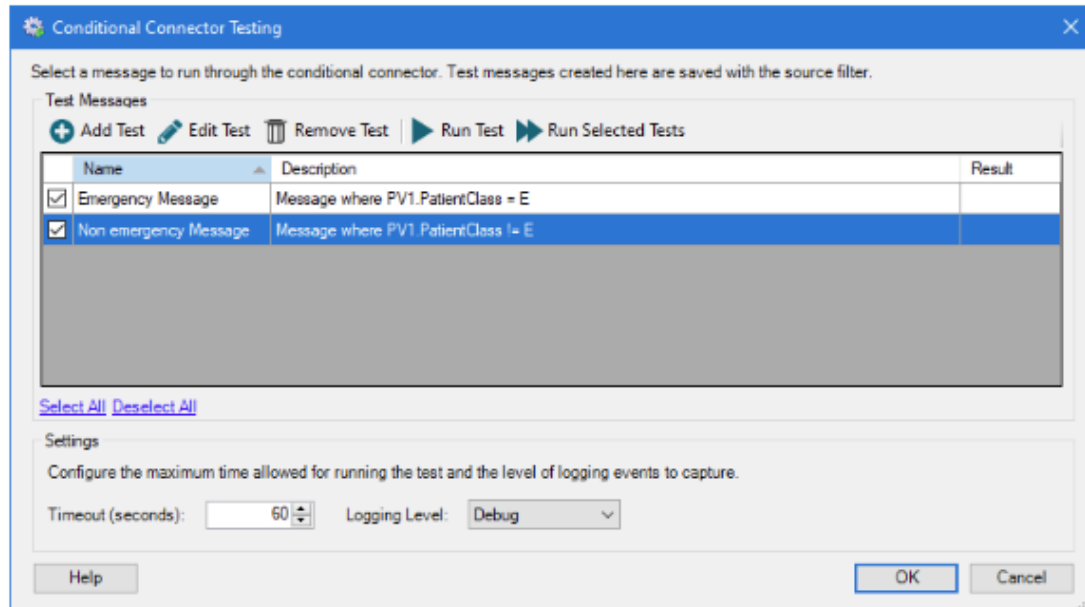
6

Click **OK** to close the **New Test Message** window.

7

Repeat this process to add the **ADTA03-1.edi** test message.

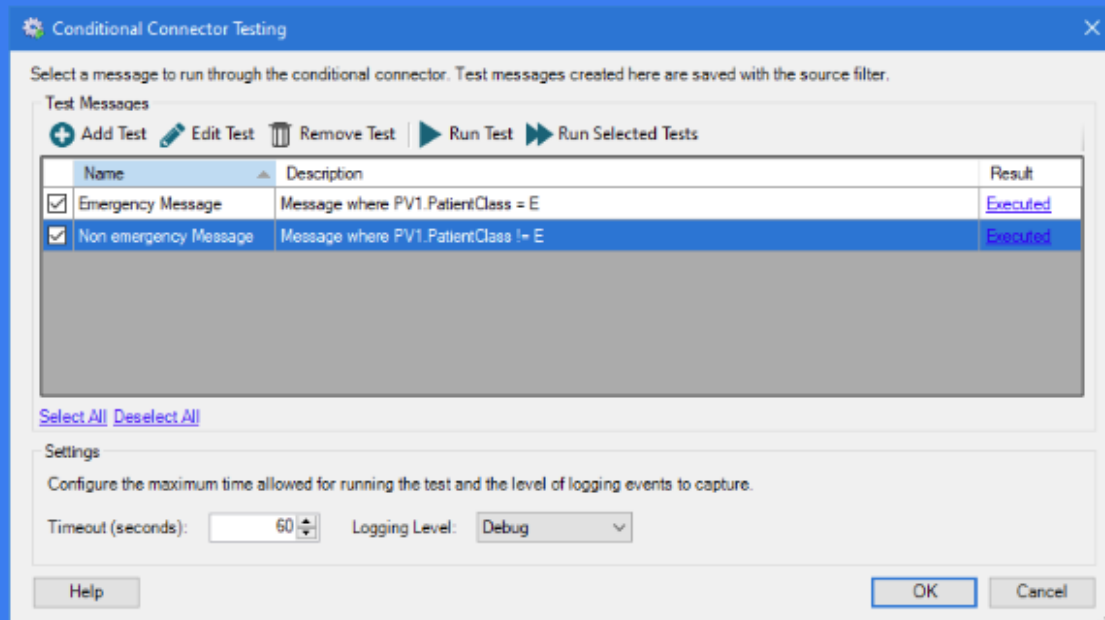
When you have finished, the window should look like the screenshot below:



One at a time, select each test message, then click **Run Test**. The expected outcome is shown in the Model Answer.

Model Answer

The results of running the conditional connector test should look like the following:



The **Executed** result shows that Rhapsody completed the tests of the conditional connector's configuration without error.

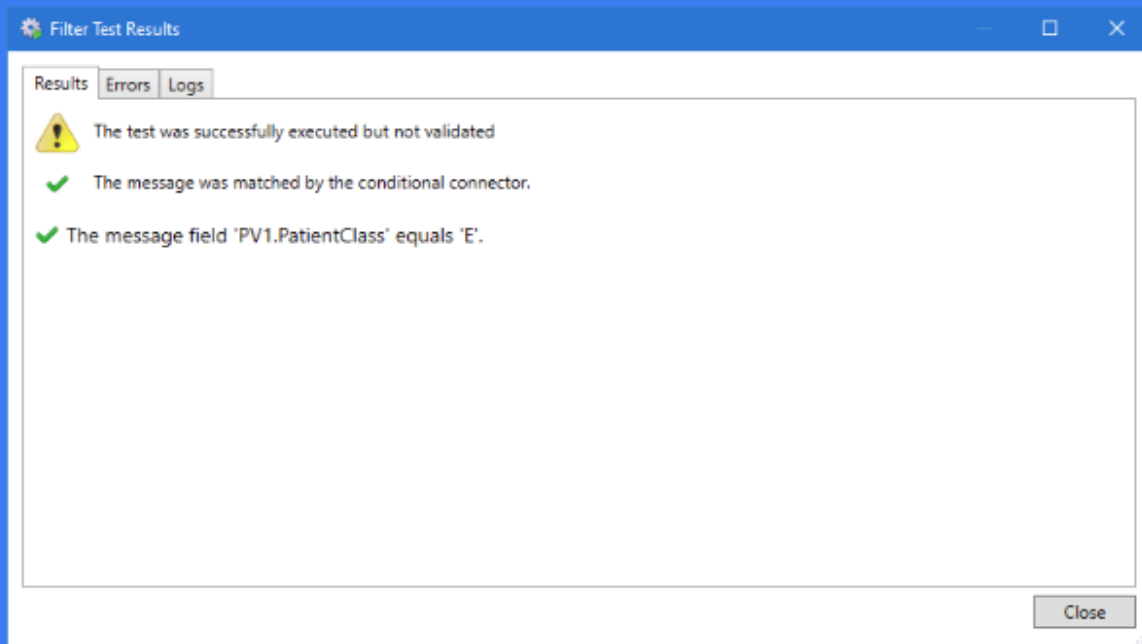
Message matched by conditional connector

The result of testing the Emergency test message against the conditional connector's configuration is shown below.

The **Executed** result shows that Rhapsody completed the tests of the conditional connector's configuration without error.

Message matched by conditional connector

The result of testing the Emergency test message against the conditional connector's configuration is shown below.



Message Not Matched by Conditional Connector

The result of testing the ADTA03 test message against the conditional connector's configuration is shown below. As this message doesn't match the rules in the conditional connector it will be sent to the filter's no match connector.



Rhapsody Variables

Rhapsody Variables allow you to specify environment-specific parameters such as port numbers, host names and directory paths. Their usage is global, which means a variable can be used simultaneously in the configuration of any communication point, filter or conditional pathway in any folder or locker on the engine.

✎ Use variables wherever a configuration field might be expected to change between environments.

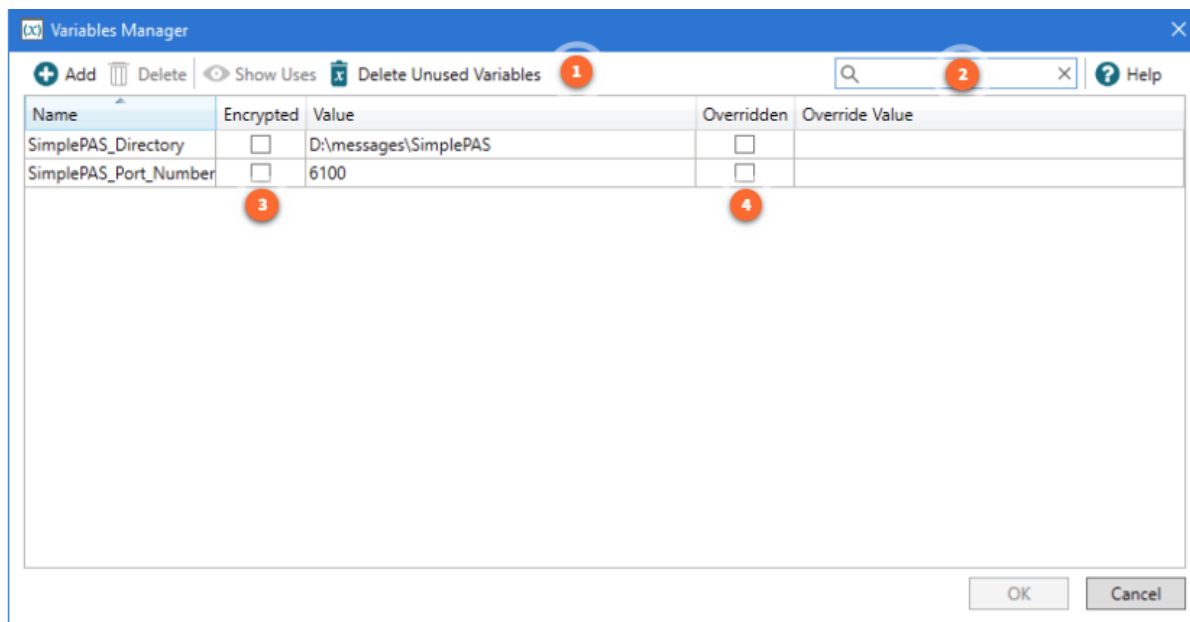
Variables are specified using the notation: `$(<VariableName>)`, for example `$(SimplePAS_Port_Number)`. The format for Rhapsody Variables should first identify the system then the component. For example, `SimplePAS_Port_Number` indicates the Port number used to connect to the SimplePAS system. This ensures that related variables are grouped together in Rhapsody.

Some component properties can be specified using a combination of a variable and a hard-coded value; for example the directory paths `$(SimplePAS_Directory)/archive` and `$(SimplePAS_Directory)/errors`. This allows a small number of variables to be used across a configuration.

⚠ Message Properties `$Property` and Rhapsody Variables `$(Variable)` appear similar when they are used in your configuration. While learning to use Rhapsody, you will need to be careful not to confuse the two concepts.

The Variables Manager

The **Variables Manager** is located under the **Managers** drop-down and on the **View** menu.



Add a variable

1. 1

Select **Add** and type the name of the new variable; do not use spaces.

2. 2

Select the **Value** field and type the value to be associated with the variable. Repeat this process for all the variables you want to create.

3. 3

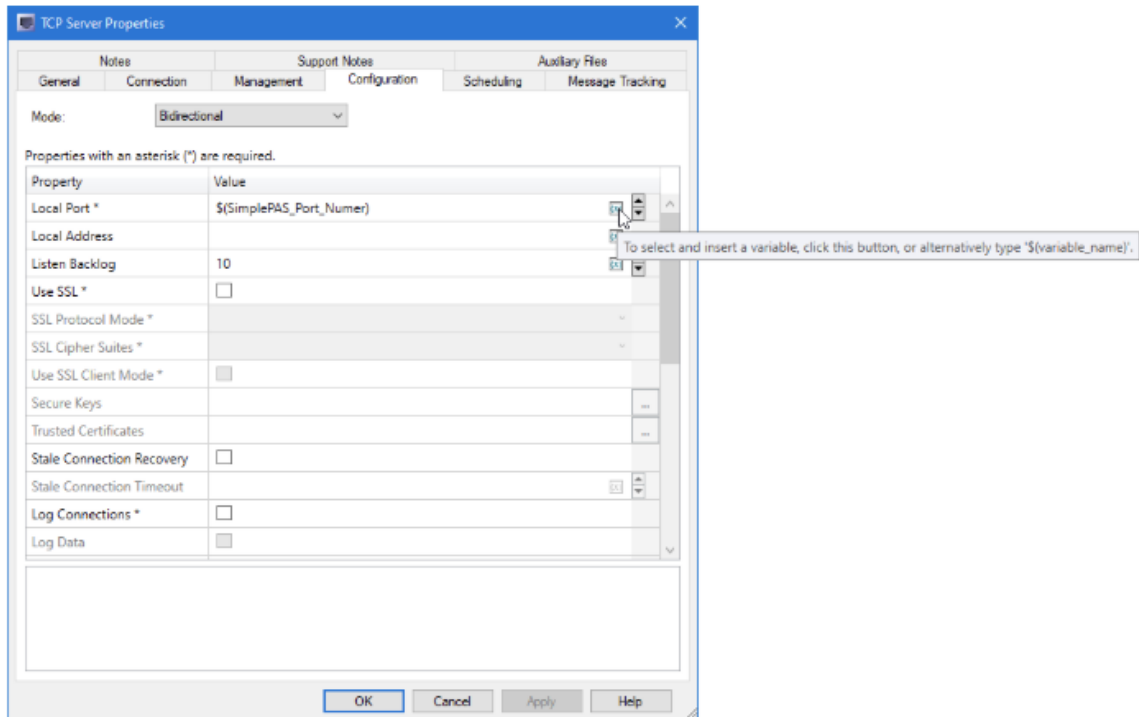
Select **OK** to check in your changes.

4. 4

Close the **Variables Manager** window. Changes to the variables will take effect as soon as they are checked in.

Insert a variable

Select the variable selector icon (shown below) in the **Value** field to select and insert a variable for a specific property.



Communication Point dialog showing mouse hovering over variable selector box.

Import Rhapsody variables

Rhapsody variables typically define values specific to a particular host or environment. When a Rhapsody configuration is migrated between environments, values specified as variables can be amended in the **Variables Manager** rather than opening each communication point or filter and changing their configuration individually.

When the contents of a RLC file is imported, Rhapsody will check for existing variables with the same name:

- If no matches are found, the variables contained in the RLC file will be added to the **Variables Manager**.
- If a match is found, the existing values are retained and the values in the RLC file will be discarded.